

Swoplet

Milestone 1: Use Cases, High-Level Requirements, and Architecture

Master Team Project / GDSD Summer 2025
Team 6

Team Lead Email: hammad.asif@informatik.hs-fulda.de

Team Members	Role
Hammad Asif	Team Lead
Karan Patel	Github Master
Pooja Puthu Vayal	Backend Lead
Akhil Sajan	Frontend Lead
Hamza Butt	Frontend Developer
Hasara Koralege	Backend Developer

History

Date	Version	Notes
08/05/2025	First Draft	Initial Submission

1. Executive Summary

At Fulda University of Applied Sciences, students and staff frequently encounter obstacles when trying to buy, sell, or exchange everyday items within a trustworthy local network. Public online marketplaces often fall short, exposing users to privacy risks, scams, and unreliable interactions with strangers outside the university. These challenges not only discourage sustainable practices like reuse and sharing but also hinder the development of a connected, supportive campus community.

To address these issues, we have launched Swoplet, a dedicated, web-based platform designed exclusively for the Fulda University community. Access is strictly limited to students, faculty, and staff with a valid `hs-fulda.de` email address, ensuring that every participant is a verified member of the university. This closed, secure environment eliminates the uncertainties of public marketplaces, offering a reliable space where users can confidently engage in peer-to-peer transactions. The platform's intuitive interface makes posting, searching, and communicating effortless, empowering users to participate in a safe and supportive campus marketplace.

What sets Swoplet apart are its innovative features, crafted specifically for the Fulda student lifestyle. Users can easily browse or list items with smart filters, searching by category, price, condition, and availability to find exactly what they need. The secure in-app messaging system allows for direct, private communication between buyers and sellers, while all listings are carefully moderated before publication to maintain quality and trust. Community safety is further enhanced by a robust reporting feature, enabling users to flag inappropriate or suspicious content.

Additional highlights include Wishlist tracking, allowing users to monitor items they're interested in; product comparison tools to help make informed decisions; and a flash deal system that promotes limited-time offers, adding excitement and urgency to the marketplace. The platform's responsive design ensures a seamless experience on laptops, tablets, and smartphones, making it convenient for users whether they're on campus or on the move.

Swoplet is developed by a passionate, student-led team at Fulda University of Applied Sciences. Drawing on firsthand experience with the challenges faced by their peers, the team combined technical expertise with a deep understanding of campus life to create a platform tailored to the unique needs of the Fulda community. Their collaborative approach ensured that every feature was designed with user experience, safety, and convenience in mind.

By providing a secure, user-friendly, and feature-rich marketplace exclusively for Fulda University, Swoplet fills a critical gap in campus life. It not only encourages sustainable habits and resource sharing but also strengthens the bonds within the university community. Ultimately, the platform empowers students and staff to connect, collaborate, and thrive in a trusted environment, making Fulda University an even better place to live, learn, and grow.

2. Personae and Main Use Cases

2.1 Persona 1: Lina – Student Seller (Hochschule Fulda)

Relevant Functional Requirements: FR1, FR2, FR3, FR4, FR5, FR10, FR15, FR20

General Characteristics: Lina is a 22-year-old computer science student at Hochschule Fulda. She lives on campus and frequently sells used textbooks and dorm essentials. She's comfortable with digital platforms and prefers mobile-friendly interfaces.

Goals:

- Sell her items quickly and easily.
- Update or temporarily hide listings as needed.
- Ensure buyers are from her university community.

Pain Points:

- Wasting time reposting expired listings manually.
- Difficulty managing multiple listings.
- Insecure selling experiences on public marketplaces.

2.2 Persona 2: Alex – Budget-Conscious Buyer (SFSU Exchange Student)

Relevant Functional Requirements: FR1, FR6, FR7, FR9, FR14, FR16, FR18, FR19

General Characteristics: Alex is a 25-year-old exchange student from SFSU spending one semester in Fulda. He needs basic furniture and appliances but has a tight budget.

Goals:

- Search, filter, and sort listings quickly.
- Track price drops and find deals.
- Save interesting listings for later comparison.

Pain Points:

- Inaccurate or outdated listings.
- Unclear communication with sellers.
- Listings that don't reflect current prices or availability.

2.3 Persona 3: Sarah – Platform Admin (Graduate Assistant)

Relevant Functional Requirements: FR11, FR12, FR13, FR22, FR23

General Characteristics: Sarah is a graduate assistant tasked with moderating the platform. She works part-time and needs efficient tools to keep the platform clean and compliant.

Goals:

- Approve or reject listings quickly.
- Address inappropriate content or user reports.
- Respond to support queries and maintain platform integrity.

Pain Points:

- High volume of spammy or incomplete posts.
- Limited moderation tools on other platforms.
- Repetitive support questions from users.

2.4 Main Use Cases

UC1 – Create and Manage a Listing (FR1, FR2, FR3, FR4, FR5, FR10, FR15, FR20)

Lina logs into the platform using her university email (FR1). She updates her profile with a photo and short bio (FR2). She wants to sell a microwave, so she clicks on

“Post New Item,” fills in the title, description, price, and category (FR3, FR5), and uploads four photos (FR4). After the item is approved by an admin (FR11), the listing becomes public. A week later, she updates the description and changes the price (FR10). She later hides the listing temporarily while out of town (FR20). If no sale happens, it auto-expires after 30 days (FR15).

UC2 – Discover, Watch, and Buy Items (FR6, FR7, FR9, FR14, FR16, FR18, FR19)

Alex visits the site to find a secondhand bike. He searches “bike” and applies a filter for listings under €100 (FR6, FR7). He bookmarks two options in his Watchlist (FR9). One listing has a visible price history showing a drop from €120 to €90 (FR18), and it's marked in the “Price Drops” section (FR19). After buying, Alex leaves a 5-star review for the seller (FR14). He shares another listing link with his roommate via WhatsApp (FR16).

UC3 – Safe Buyer-Seller Communication (FR1, FR8, FR14, FR24)

After seeing a listing for a used monitor, Alex messages the seller directly through the platform without sharing emails (FR8). They negotiate price and pick-up time. After purchase, both users leave a review and rating for each other (FR14). Later, Alex deletes several old conversations to clean his inbox (FR24).

UC4 – Admin Moderation and User Support (FR11, FR12, FR13, FR22, FR23)

Sarah logs into the admin dashboard and sees 18 pending listings (FR11). She rejects one listing due to offensive content and bans the user for repeated violations (FR12). She reviews a flagged item reported by another user (FR13) and takes appropriate action. She then checks the support inbox where users submitted questions via the Contact Form (FR22). Finally, she updates the FAQ section with new answers based on recurring issues (FR23).

UC5 – User Registration and Recovery (FR1, FR17, FR21)

New student Ahmad wants to join the platform. He registers using his hs-fulda.de email (FR1) and is required to accept the Terms of Service and Privacy Policy (FR17). A few weeks later, he forgets his password and uses the reset option to set a new one (FR21). He logs back in, updates his profile, and begins browsing.

3. Main Data Items and Entities

3.1 User Types and Privileges

User Type	Description	Privileges
Guest	Unregistered visitor	View listings, apply filters
Buyer	Verified Fulda user (buyer)	Register/login, search/filter, message, favorites, compare, report
Seller	Verified Fulda user (seller)	Verified Fulda user (seller) CRUD listings, upload media, respond to messages, temporary hides, report
Moderator	System user with moderation privileges	Approve/reject listings, manage reports, monitor messages, warn/ban users, manage expirations

3.2 Main Data Entities and Descriptions

Entity Name	Description
User	Registered individual (Guest, Buyer, Seller, Moderator)
Product Listing	Item for sale; includes title, description, price, condition, images, etc.
Media File	Images/videos uploaded and moderated
Message	In-app chat communication between users
Favorite	Saved product for later comparison

Report	Complaint against a listing or user
Search History	Log of user search queries and filters
Price Comparison	Table for comparing similar products
Time Limit	Listing expiration settings

3.3 Logical Data Structure (High-Level Overview)

1. User:

- user_id
- name
- email
- password_hash
- role (guest, buyer, seller, moderator)
- profile_picture
- registered_at
- is_active

2. Product Listing:

- product_id
- title
- description
- category
- price
- condition (new, used)
- tags (electronics, laptop, HP)
- location
- created_by_user_id
- status
- created_at
- updated_at

3. Media File:

- media_id
- product_id
- file_path
- is_approved
- uploaded_at

4. Message:

- message_id
- sender_id
- receiver_id
- product_id
- content
- created_at

5. Favorite:

- favorite_id
- user_id
- product_id
- saved_at

6. Report:

- report_id
- reporter_id
- target_type (user, listing)
- target_id
- reason
- Status (pending, reviewed, resolved)
- created_at

7. Search History:

- search_id
- user_id
- filters_used
- timestamp

8. Time Limit:

- time_limit_id
- product_id
- start_time
- end_time
- status (active, expired)

4. Functional Requirements

FR1. User Registration & Affiliation Check

Users must register using a verified university email (e.g., hs-fulda.de) to ensure access is limited to Fulda/SFSU affiliates.

FR2. User Profile Management

Users should be able to view, edit, and update their profile details, including name, photo, and short bio.

FR3. Post New Item/Service

Authenticated users can create listings for products or services by entering a title, description, price, and other details.

FR4. Upload Media for Listing

Users can upload multiple images (or a video) for each listing to showcase their item or service.

FR5. Categorize Listings

Users can select categories and tags while posting, to help others find listings more easily.

FR6. Browse and Search Listings

Users can search listings by keyword or browse through categorized sections.

FR7. Filter and Sort Listings

Users can filter search results by category, price range, tags, etc., and sort by newest, oldest, or price.

FR8. Internal Messaging

Buyers and sellers can communicate through an in-platform messaging system without sharing personal emails.

FR9. Add to Watchlist

Users can save listings to a personal watchlist to revisit them later.

FR10. Seller Dashboard

Sellers can view and manage their listings, including edit/delete actions and listing status (e.g., active, expired, sold).

FR11. Admin Approval for Listings

All posted listings must be reviewed and approved by an admin before becoming publicly visible.

FR12. Admin Moderation Capabilities

Admins can remove inappropriate listings or ban users when necessary.

FR13. Report Inappropriate Listings

Users can flag listings or messages as inappropriate to alert admins for review.

FR14. Flash Sale Listing

Allow sellers to mark a listing as a “Flash Sale,” highlighting it prominently for a limited time to quickly attract buyers—ideal for urgent situations like moving out.

FR15. Automatic Listing Expiry

Listings will automatically expire after a set period (e.g., 30 days), with an option to renew.

FR16. Share Listings Externally

Users can share listing URLs via social media, email, or other messaging platforms.

FR17. Accept Terms and Privacy Policy

Users must accept the site’s Terms of Service and Privacy Policy during registration.

FR18. Track Price History

Each listing will show a basic price change history so users can see if the price has been reduced over time.

FR19. Highlight Discounted Listings

Display listings whose prices have dropped from their original value in a special “Price Drops” section to catch buyer attention.

FR20. Temporarily Hide Listing

Sellers can temporarily pause a listing from public view without deleting it—useful for restocking or updating.

FR21. Password Reset Flow

Enable users to reset forgotten passwords and set new ones to regain access to their account.

FR22. Contact/Support Form

A built-in “Contact Us” form for users to submit questions or issues, routing entries to a monitored support inbox.

FR23. FAQ/Help Page

Provide a static FAQ section covering common questions (e.g., how to post, how to flag content) so users can self-serve basic support.

FR24. Bulk Message Deletion

In the messaging area, let users select and delete multiple old conversations at once for inbox cleanup.

5. Non-Functional Requirements

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in Milestone 0. Application delivery shall be from chosen cloud server
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers
3. All or selected application functions must render well on mobile devices
4. Data shall be stored in the database on the team's deployment cloud server.
5. No more than 50 concurrent users shall be accessing the application at any time
6. Privacy of users shall be protected and all privacy policies will be appropriately communicated to the users.
7. The language used shall be English (no localization needed)
8. Application shall be very easy to use and intuitive
9. Application should follow established architecture patterns
10. Application code and its repository shall be easy to inspect and maintain
11. Google analytics shall be used (optional for Fulda teams)
12. No email clients shall be allowed.
13. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
14. Site security: basic best practices shall be applied (as covered in the class) for main data items
15. Application shall be media rich (images, video etc.). Media formats shall be standard as used in the market today

16. Modern SE processes and practices shall be used as specified in the class, including collaborative and continuous SW development
17. For code development and management, as well as documentation like formal milestones required in the class, each team shall use their own GitHub to be set-up by class instructors and started by each team during Milestone 0
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "Fulda University of Applied Sciences Software Engineering Project, Summer 2025 For Demonstration Only" at the top of the WWW page. (Important so as to not confuse this with a real application).

6. Competitive Analysis

Feature	eBay Kleinanzeigen	Facebook Marketplace	Shpock	Swoplet
Account creation & profile	✓	✓	✓	✓
Post/edit/delete listings	✓	✓	✓	✓
Advanced search & filter	Basic filters	Moderate	✓	Custom filters
In-site messaging	✓	✓	✓	✓
Admin approval for listings	✓	✓	✓	✓
Favorites/bookmarked listings	✓	✓	✓	✓
Time-limited listings	✗	✗	Premium	✓
Report users/listings	✓	✓	✓	✓
Role-based access control	✓	✓	✓	✓

University-only access	✗	✗	✗	✓ (hs-fulda.de)
Compare	✗	✗	✗	✓

7. High-Level System Architecture

7.1 Architecture Overview

Our application follows a three-tier architecture consisting of the Presentation Layer (Frontend), Application Layer (Backend), and Data Layer (Database), all hosted on a cloud-based infrastructure.

Frontend (Presentation Layer)

- Developed using React.js for a dynamic and responsive Single Page Application (SPA).
- Styled using Tailwind CSS, enabling fast and consistent UI design.
- Uses RESTful for communicating with the backend API.
- Hosted behind NGINX, which serves the static frontend files and proxies API requests to the backend.

Backend (Application Layer)

- Built with Node.js and Express.js, providing a lightweight and efficient REST API.
- Handles all core logic such as user authentication, listing management, messaging, and admin approvals.
- Implements middleware for validation, role-based access control, and error handling.
- Ensures secure communication using SSL/TLS encryption.

Database (Data Layer)

- *Uses MySQL 8.0 as the relational database.*
- *Stores structured data related to users, listings, messages, reviews, and admin records.*

- *Connected securely via internal networking and accessed only from within the cloud server.*

Web Server & Hosting

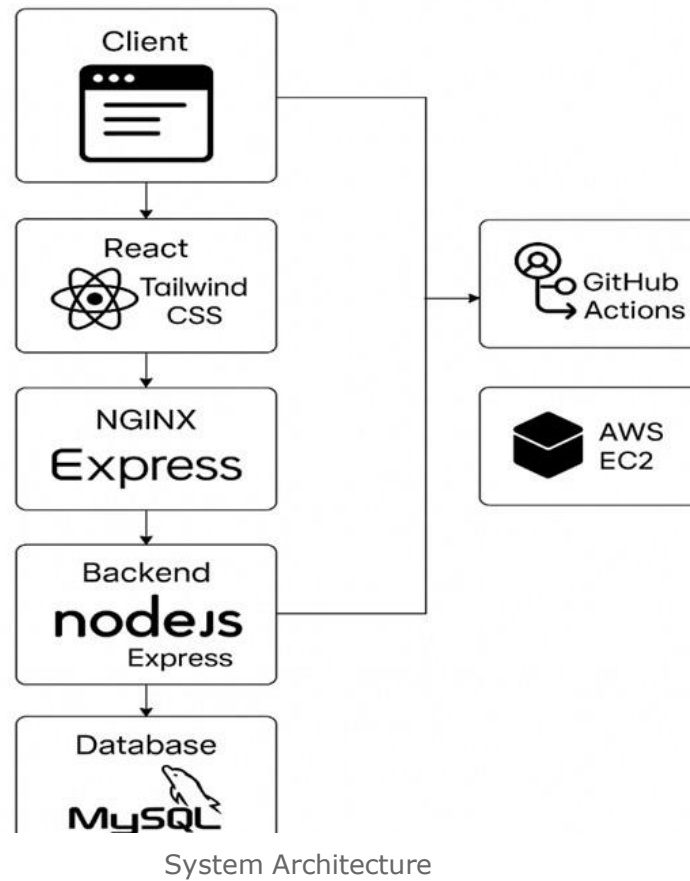
- NGINX is configured as a reverse proxy, serving frontend files and forwarding API traffic.
- The entire application stack is hosted on AWS EC2 (t2.micro) running Ubuntu 22.04 LTS.

CI/CD & Deployment

- GitHub Actions is used for Continuous Integration and Deployment.
- Code pushed to the main branch triggers an automated build and deployment to AWS.
- All environment variables and credentials are securely managed through AWS and GitHub secrets.

Security Measures

- HTTPS via Let's Encrypt.
- Input validation and sanitization to prevent SQL injection and XSS attacks.
- Admin approval system for listings and media moderation.



8. Team and Roles

Team Members	Role
Hammad Asif	Team Lead
Karan Patel	Github Master
Pooja Puthu Vayal	Backend Lead
Akhil Sajan	Frontend Lead
Hamza Butt	Frontend Developer

9. Checklist

1. The team found a time slot to meet (online) outside of the class. - DONE
2. GitHub master chosen. - DONE
3. The team decided and agreed together on using the listed SW tools and deployment
4. server. - DONE
5. Team ready and able to use the chosen back and front-end frameworks and those
6. who need to learn are working on learning and practicing. - ON TRACK
7. The team lead ensured that all team members read the final M1 and
8. agreed/understood it before submission. - DONE
9. GitHub is organized as discussed in class. - DONE

SWOPLET

MASTER TEAM PROJECT SOSE 2025 MILESTONE 2: MORE DETAILED REQUIREMENTS, SPECS, ARCHITECTURE, UI MOCK-UPS AND VERTICAL SW PROTOTYPE

Team Project / GDSD Summer 2025 Team 6

Team Lead Email: hammad.asif@informatik.hs-fulda.de

TEAM MEMBERS

Team Member	Role
Hammad Asif	Team Lead
Karan Patel	Github Master
Pooja Puthu Vayal	Backend Lead
Akhil Sajan	Frontend Lead
Hamza Butt	Frontend Developer

HISTORY

Date	Version	Notes
29/05/2025	First Draft	Initial Submission

1. EXECUTIVE SUMMERY

University students, teachers, and employees in this competitive era of time frequently struggle to buy, sell, and exchange goods within their educational community. The existing platforms are mostly public-oriented, and this can cause problems in security, authenticity, and relevance. In addition, users get lost in setting fair prices for products, resulting in poor bargaining and missed opportunities. For Fulda University student association these concerns delay the capability of having a credible, available, and efficient marketplace that meets their specific needs.

To address these challenges, Fulda University Masters team is developing a focused web-based site. The site will be restricted to serving the Fulda University exclusively, allowing students, faculty members, and staff to list things for sale securely, search ads, and complete transactions. Key features include:

- Listing items for sale with complete descriptions and photos.
- Instant searching and browsing of items available for easy selection through basic filters.
- Restriction of user interactions to members of only the involved university communities through a secure messaging system.
- Admin moderation to enable posts to go live once approved and for removing offending content or users, ensuring quality and safety.

Besides the basic features, the platform is built in a flexible way that allows it to continuously evolve and add new, creative, and top-quality features.

Among the platform's unique and competitive features is the integration of an Artificial Intelligence (AI) model that analyzes market data to provide buyers with suggestions on the average market price of similar products. This feature empowers users with valuable pricing insights, enabling sellers and buyers to negotiate more wisely and more fairly. Through the use of AI-driven price estimating, the platform not only enhances transaction transparency but also gets a significant advantage over existing marketplaces, acquiring and maintaining users through the provision of tangible value to the shopping experience.

The application is developed by a dynamic group of about five students who closely collaborate under the instructions of the CTO. The diverse background of the team and Agile Systems Engineering approach enable them to iteratively evolve the functionality of the platform to maximize the fulfillment of user needs. With careful milestones, the team is endeavoring to continuously monitor and enhance the service to offer robustness, security, and user satisfaction. Lastly, the objective is to develop a highly respected, vibrant online marketplace dedicated to the Fulda University community and providing superior value while building a positive campus commerce environment.

2. FUNCTIONAL REQUIREMENTS ALONG WITH PRIORITY

2.1 PRIORITY 1 - MUST HAVE FUNCTIONAL REQUIREMENTS

2.1.1 All Users

- **FR1 – User Registration& Affiliation Check:** Users must register using a verified institutional email (e.g., @hs-fulda.de or @sfsu.edu). This ensures that only affiliated members can access the platform.
- **FR6 – Browse and Search Listings:** Users should be able to explore available items/services and use search functionality with keywords to locate specific listings.
- **FR7 – Filterand Sort Listings:** Users can narrow down results using filters (e.g., category, price, tags) and sort by parameters like newest, oldest, or price (low-high, high-low).

2.1.2 Buyer

- **FR8 – Internal Messaging:** Enables buyers to communicate securely with sellers through a built-in messaging system, without sharing personal contact details.
- **FR9 – Wish listing a Product:** Buyer can browse and wish list a product for future use.

2.1.3 Seller

- **FR3 – Post New Item/Service:** Authenticated sellers can create listings by entering essential details like title, description, category, price, and contact preferences.
- **FR4 – Upload Media for Listing:** Sellers can add multiple images or videos to visually showcase the product or service they are offering.
- **FR8 – Internal Messaging:** Sellers can respond to buyer inquiries through the platform's internal messaging system.
- **FR10 - Seller Dashboard:** A dedicated dashboard allows sellers to view and manage their listings (edit, delete, pause, mark sold) and track engagement.
- **FR20 - Temporarily Hide Listing:** Sellers can temporarily pause a listing from public view without deleting it; useful for restocking or updating
- **FR5 - Categorize Listings:** Sellers must select appropriate categories and add relevant tags to ensure their listings are easily discoverable

2.1.4 Admin

- **FR11 – Admin Approval for Listings:** Every listing must be reviewed and approved by a moderator/admin before becoming publicly visible, ensuring quality and safety.
- **FR12 – Admin Moderation Capabilities:** Moderators can remove inappropriate content, block or ban abusive users, and maintain platform standards.

2.2 PRIORITY 2 - DESIRED FUNCTIONAL REQUIREMENTS

2.2.1 Buyer

- **FR2 - User Profile Management:** Users should be able to view, edit, and update their profile details, including name, photo, and short bio. •
FR26 – Rate Seller: Buyer can post a rating for seller
- **FR13 - Report Inappropriate Listings:** Users can flag inappropriate listings/ messages for admin review.
- **FR24 – AI-Powered Market Price Estimation :** The system shall integrate an AI model that analyzes historical and real-time market data of similar product listings to provide users with an estimated average market price.

2.2.2 Seller

- **FR19 – Highlight Discounted Listings:** Listings with reduced prices are automatically added to a “Price Drops” section for greater visibility and buyer engagement.
- **FR2 - User Profile Management:** Users should be able to view, edit, and update their profile details, including name, photo, and short bio.

2.3 PRIORITY 3 - OPPORTUNISTIC FUNCTIONAL REQUIREMENTS

2.3.1 Users (Seller, Buyer):

- **FR17 - Accept Terms and Privacy Policy:** Users must accept the Terms of Use and Privacy Policy during account creation. •
FR16 - Share Listings Externally: Users can copy or share the listing link via social platforms or email.

- **FR21 - Password Reset Flow:** Users can reset forgotten passwords through an email-based recovery system.
- **FR23 - FAQ/Help Page:** A static help page with answers to common questions (e.g., how to post, flag, contact seller).

2.3.2 Buyer:

- **FR18 - Track Price History:** Shows a small log of previous prices to help buyers see if the price has dropped.

2.3.3 Seller:

- **FR14 - Flash Sale Listing:** Sellers can mark items as “Flash Sale” for urgent attention (e.g., moving out, quick deals).
- **FR15 - Automatic Listing Expiry:** Listings automatically expire after a set period (e.g., 30 days), with the option to renew.

2.3.4 Moderator:

- **FR13 - Report Inappropriate Listings:** Users can flag inappropriate listings/messages for admin review (admin will be responsible for reviewing the flagged content).
- **FR22 - Contact/Support Form:** A contact form routes inquiries or issues to a monitored support inbox.

3. LIST OF MAIN DATA ITEMS AND ENTITIES

3.1 USER TYPES AND PRIVILEGES

User Type	Description	Privileges
Guest	Unregistered visitor browsing the platform	• View public listings • Apply filters to search products
Buyer		• Register and log in

User Type	Description	Privileges
-----------	-------------	------------

	Verified Fulda user who buys products	• Search and filter products • Contact sellers via in-site messaging • Add products to favorites • Compare product prices • Report listings or sellers
Seller	Verified Fulda user posting products for sale	• Register and log in • Create, update, delete product listings • Upload media (images) • Respond to buyer messages • View listings • Set time limits for product availability • Report buyers
Moderator	System user with full access and moderation privileges	• Approve or reject listings • Manage flagged reports • Monitor messages for violations • Warn/ban users • Review and publish listing expiration (time limit) actions

3.2 MAIN DATA ENTITIES AND DESCRIPTIONS

Entity Name	Description
User	A registered individual on the platform (Guest, Buyer, Seller, Moderator)
Product Listing	Represents an item for sale. Includes title, description, price, condition, images, and time limits.
Media File	Images uploaded by sellers and reviewed by moderators.
Message	In-app chat communication between buyer and seller.
Favorite	Product saved by a buyer for later access and comparison.
Report	A complaint raised against a listing or user. Routed to moderators.
Search History	User-specific log of search queries and filters applied.

Entity Name	Description
Price Comparison	Dynamic table for comparing similar products by price, condition, and seller.
Time Limit	Optional setting by sellers to define listing expiration.

3.3 LOGICAL DATA STRUCTURE (HIGH-LEVEL OVERVIEW)

Users Table

Field Name	Description
user_id	Primary Key, unique user identifier
username	Display name of the user
password	Hashed user password
email	University email (@hs-fulda.de), unique
role	Refers to user role (Guest, Buyer, Seller, Moderator)
permission	Optional access control flag
profile_picture	URL to profile image
created_at	Account creation timestamp
modified_at	Last profile update timestamp
Is active	Boolean to store active status
warn_count	How many times moderator have warned

Product Listings Table

Field Name	Description
product_id	Primary Key, unique listing identifier
title	Title of the product
description	Detailed product information

Field Name	Description
category	Product category (e.g., Books, Electronics)
price	Listed price in €
condition	New or Used
tags	Optional keywords for search
location	Seller location
Created_by_id	Foreign Key referencing Users
status	Listing state (e.g., pending, approved)
created_at	Listing creation timestamp
updated_at	Last listing update timestamp

Media Files Table

Field Name	Description
Media id	Primary Key, unique media file ID
Product id	Foreign Key referencing Product Listings
file_path	Path or URL of the uploaded media
is_approved	Boolean flag for admin approval
uploaded_at	Media upload timestamp

Chat Table

Field Name	Description
chat id	Primary Key, unique chatID
product_id	Foreign Key referencing Product
product_owner_id	Foreign Key referencing Users

Field Name	Description
last_message_id	Foreign Key referencing Messages
created at	Chat timestamp

Messages Table

Field Name	Description
message_id	Primary Key, unique message ID
sender id	Foreign Key referencing Users
receiver id	Foreign Key referencing Users
product_id	Foreign Key referencing Product Listings
content	Message text
created at	Message sent timestamp

Favorites Table

Field Name	Description
favorite id	Primary Key
user id	Foreign Key referencing Users
product_id	Foreign Key referencing Product Listings
saved at	Timestamp when listing was bookmarked

Reports Table

Field Name	Description
report_id	Primary Key
reporter_id	Foreign Key referencing Users
targetType	Reported type (user or listing)

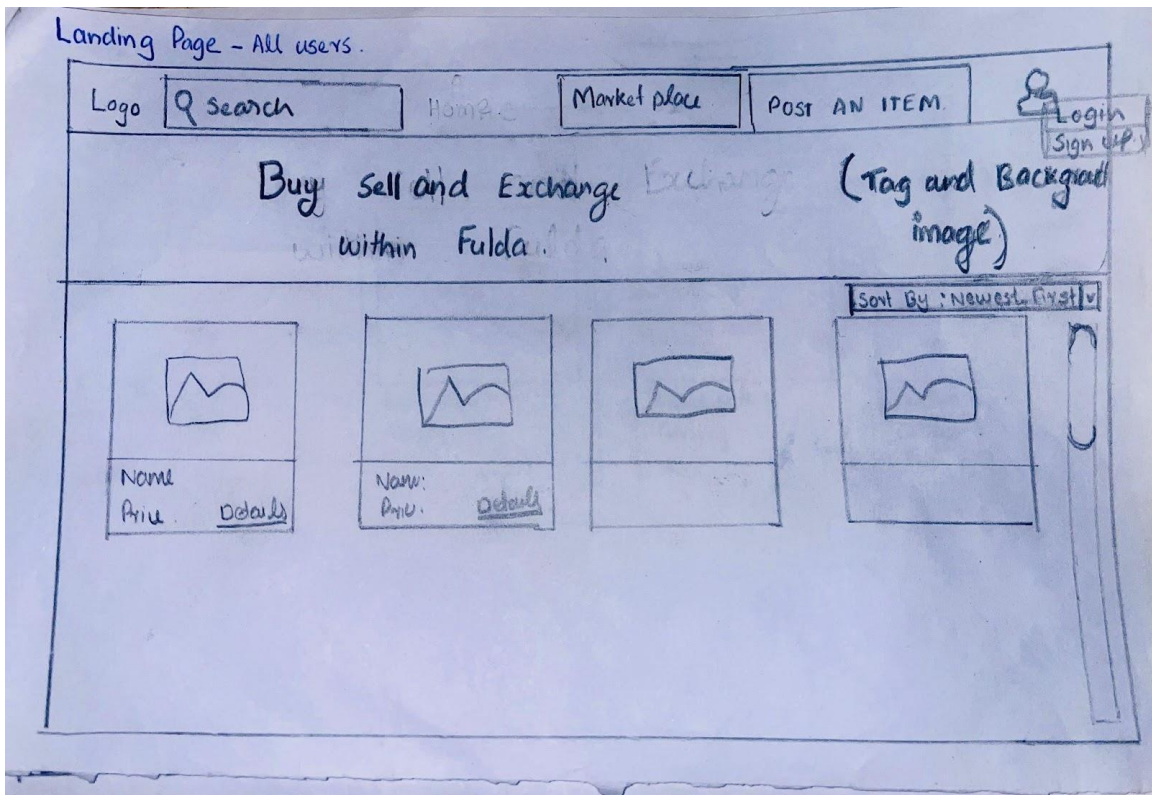
Field Name	Description
target_id	ID of the reported entity
reason	Reason for reporting
status	Report status (pending, resolved)
created at	Report submission timestamp

Search History Table

Field Name	Description
search id	Primary Key
user id	Foreign Key referencing Users
filters used	Search filters applied (stored as text/JSON)
timestamp	Time of search

4. NEW: UI MOCKUPS AND STORYBOARDS (HIGH LEVEL ONLY)

4.1 Landing Page



Description:

This is the landing page of our marketplace website, designed with a clean and engaging interface to welcome visitors. Users can browse featured product listings or choose to register and sign in as members. Once logged in, they gain full access to buy and sell items within the Fulda University community.

Key Features:

- **Main Actions:** Prominent “Browse” and “Sell” buttons for quick access to key functions. Browse button goes to marketplace and Post an Item button goes to login page. Details link of each product will lead to product detail page. But to contact a seller or add to wishlist option user needs to login.
- **Featured Listings:** A visually appealing horizontal carousel showcasing selected items (e.g., textbooks, electronics).

- **Login Section:** Streamlined login fields for returning users to sign in using their email and password.

4.2 User Login

Hand-drawn wireframe of a login page. The page is titled "Login Page" in the top left. It features a "Logo" placeholder in the top left and "Home | About Us" in the top right. The main content area contains an "Email*" label above a text input field, a "Password*" label above a text input field with a "Show" button to its right, a "Remember me." checkbox, a "Sign in" button, and a "New user?" label next to a "Sign up." button.

Description:

This login interface allows existing users to access their accounts on the Fulda University marketplace platform. The form provides a simple and secure way for users to authenticate and access personalized features.

Key Features:

- **User Authentication:** Email and password input fields for user authentication.
- **Remember me:** Checkbox for convenient future logins.

4.3 Registration Page

The wireframe shows a registration form titled "Sign-up (Registration Page)". It includes a header with "Logo" and "Home" links. The form fields are: "Full Name", "Email", "Password", and "Repeat Password", each with a corresponding input box. To the right of these fields is a box labeled "upload profile photo". At the bottom, there is a "Register" button and a link "Already a user?" next to a "Login" button.

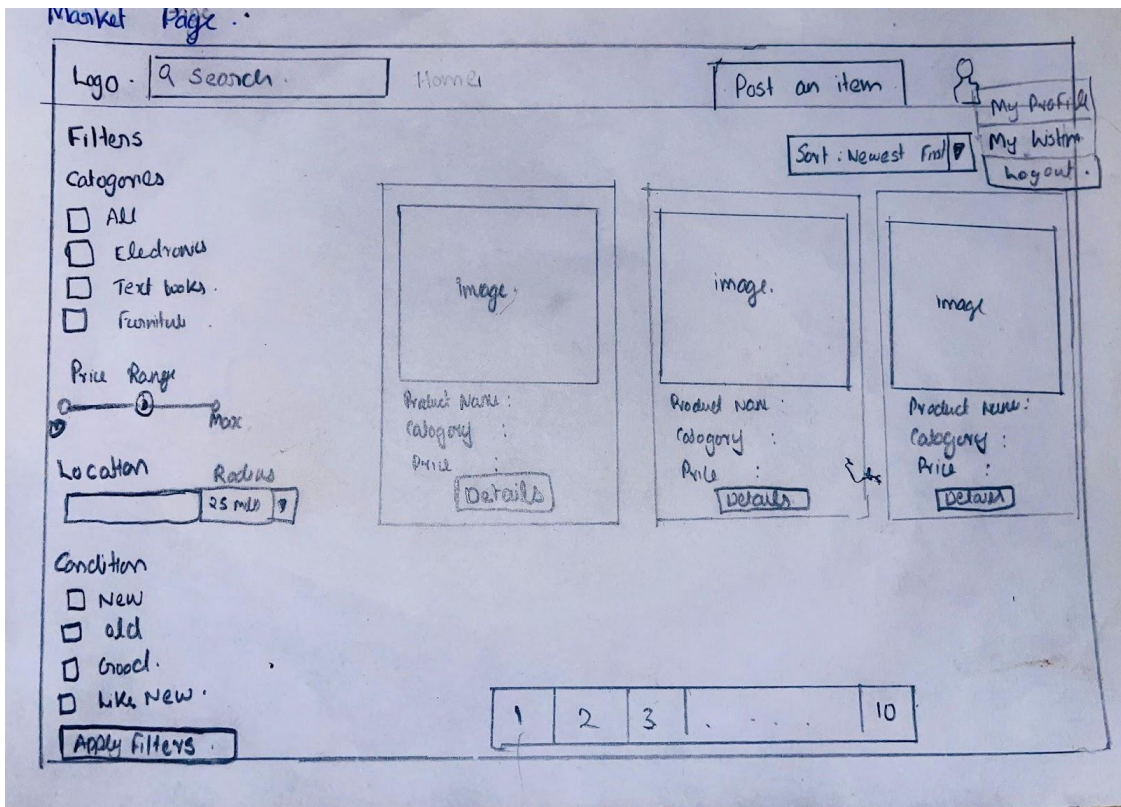
Description:

This page allows new users to sign up and become a part of the Fulda University marketplace. The form collects essential information such as full name, email, and password, and includes an image upload option to personalize user profiles.

Key Features:

- **User Information Fields:** Collects full name, email address, and password.
- **Image Upload:** Allows users to add a profile image during sign-up.

4.4 Marketplace



Description:

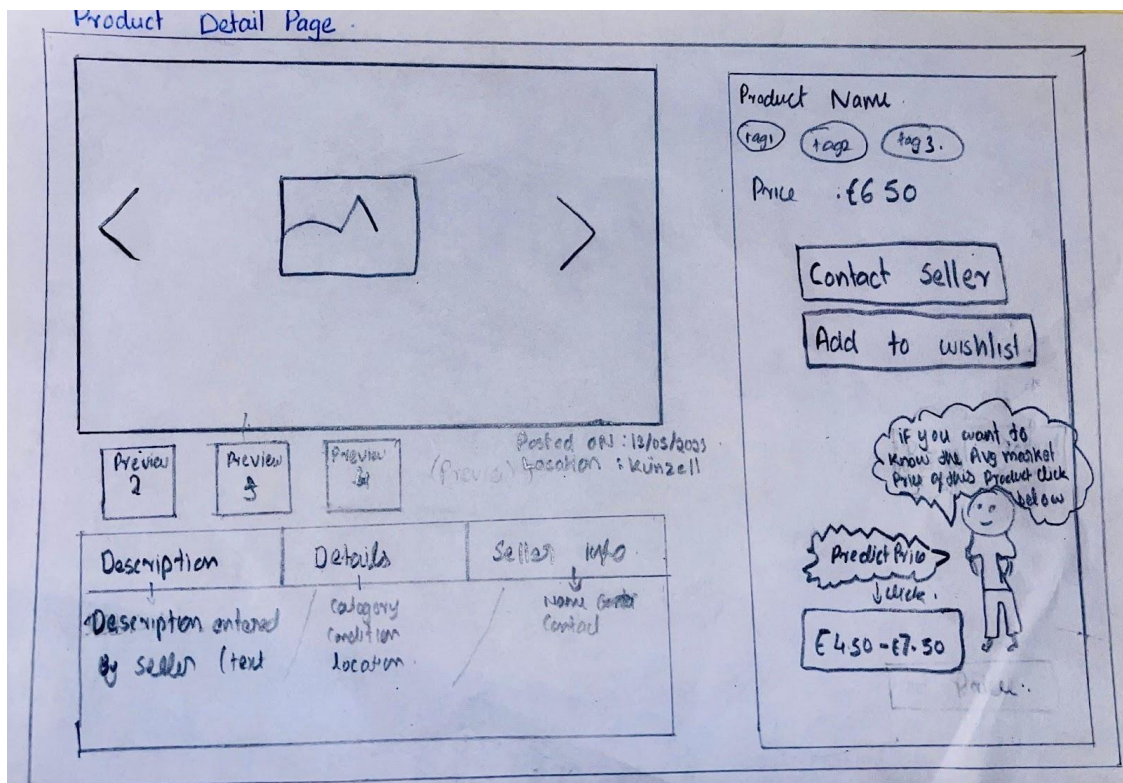
This is the main product browsing interface where users can explore items listed by other students. The marketplace is equipped with filtering and sorting features to help users quickly find relevant items across categories, price ranges, conditions and location.

Key Features:

- **Search and Filter Tools:** Search bar, category filters (e.g., Electronics, Textbooks), price range selectors, and condition checkboxes (New, Used, etc.).
- **Sorting Options:** Sort items by newest first or price drops.
- **Product Cards:** Each card includes an image, product name, category, price, and product tags.

4.5 Product Detail Page

Description:



This detailed product view provides comprehensive information about individual marketplace items. The page is designed to give potential buyers all the information they need to make informed purchasing decisions.

Key Features:

- **Product Image:** Large product image gallery with navigation arrows and thumbnail selection.
- **Product Information:** Product information including title, price, and descriptive tags, Pickup location and campus information.
- **Seller Details:** Seller contact information and rating system
- **Action buttons:** Contact Seller, Add to Wishlist
- **AI model to predict the average market price** of the product which helps buyer to negotiate.

4.6 Create Product Listing

CREATE LISTING PAGE / Edit listing

Logo · Q Search · Name · Market place · [User Icon]

Product Name/Title ·
[Text Input]

Description ·
[Text Area]

Category · [Select] ✓ Condition · [select] ✓

Price · [Text Input] Location · [] ✓

[Image Icon] [Image Icon] [Image Icon] [Image Icon] (upto 4)

POST ·

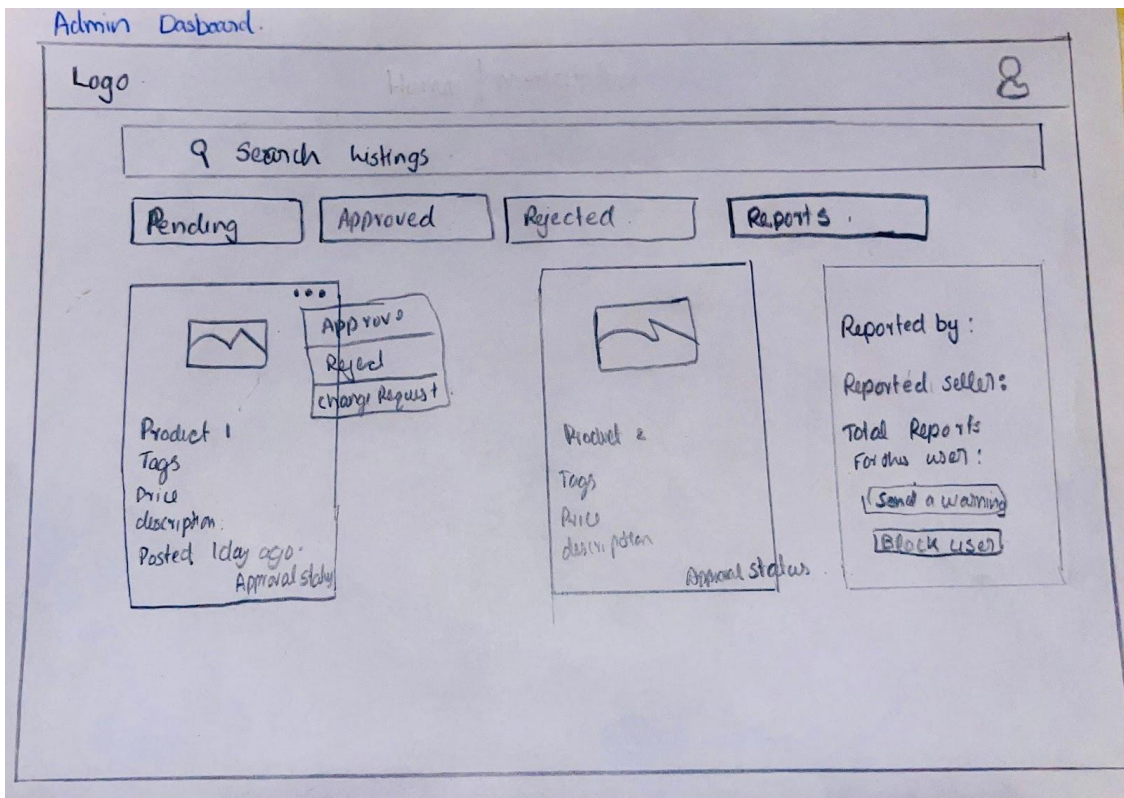
Description:

This comprehensive listing creation form allows users to add new items to the marketplace. The page provides all necessary fields and options for users to create detailed, attractive listings that will help them sell their items effectively.

Key Features:

- **Product Title:** Title input field for listing name
- **Product Description:** Large description text area for detailed item information
- **Category & Condition:** Category and condition dropdown selectors for proper classification.
- **Photo Upload and Preview:** Photo upload functionality supporting up to 4 images with upload indicators

4.7 Admin Dashboard



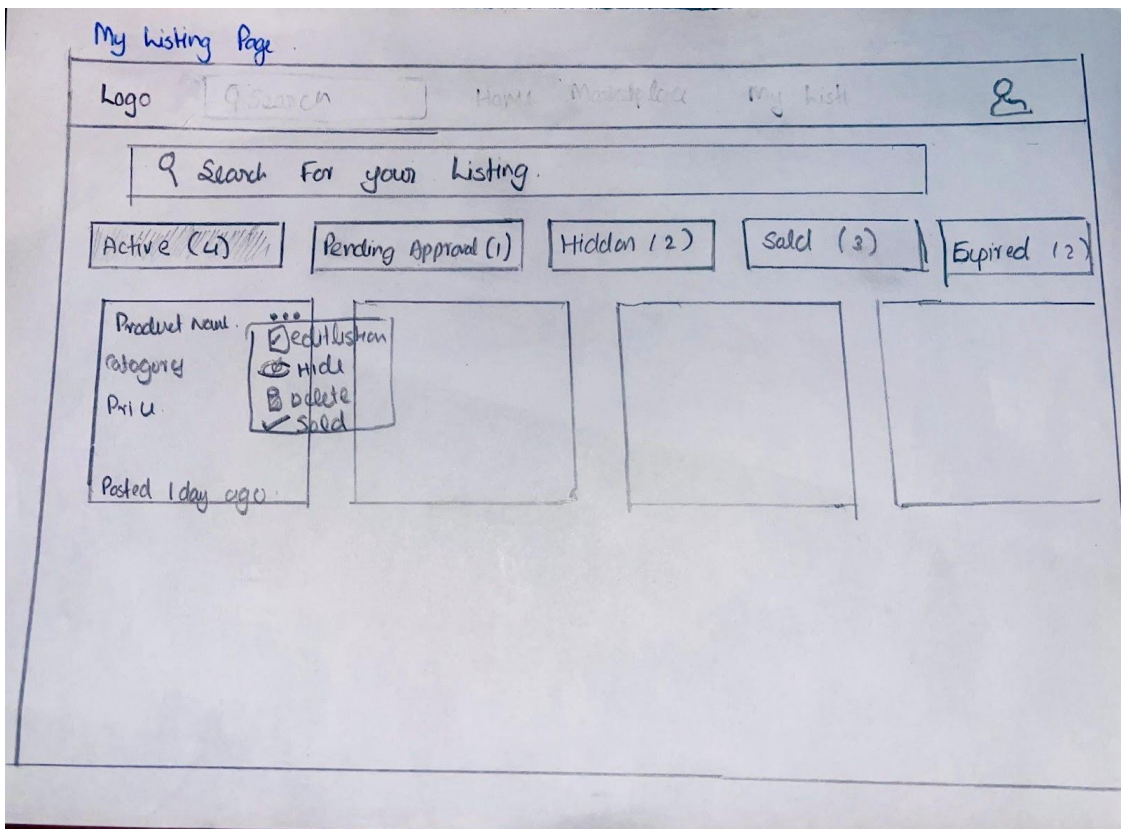
Description:

This administrative interface provides platform moderators with tools to oversee marketplace activities and manage user-submitted content. The dashboard enables efficient content moderation and platform management.

Key Features:

- **Status management tabs:** Approved, Pending, Rejected
- **Content Moderation:** Product listing cards with moderation controls
- **Action:** Approval workflow with multiple action options (Approve, Reject, Change, Request modifications)
- **Manage Reported Items:** Administrative actions (Delete, Dismiss, Report) for content management.

4.8 My Listings Page



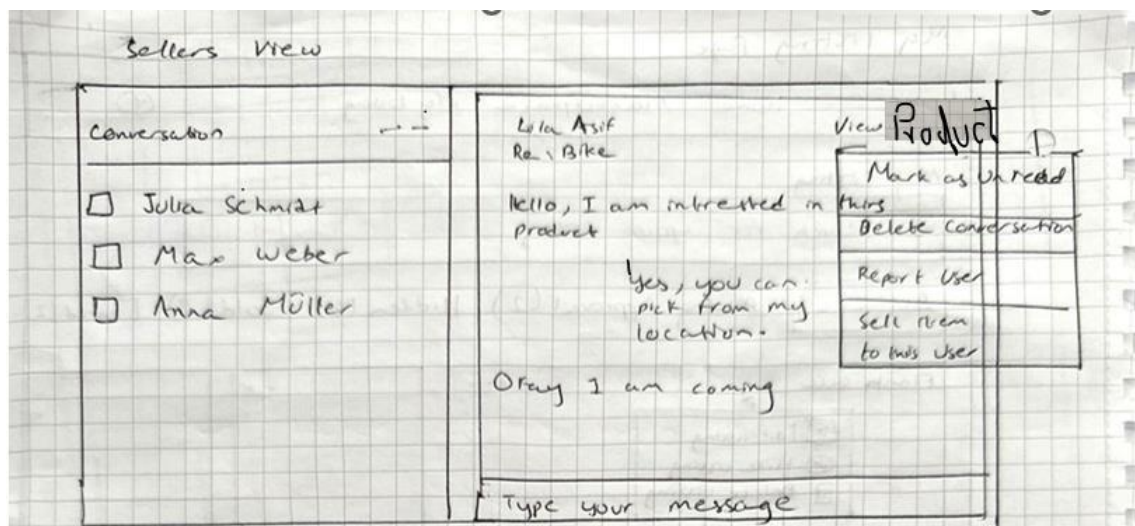
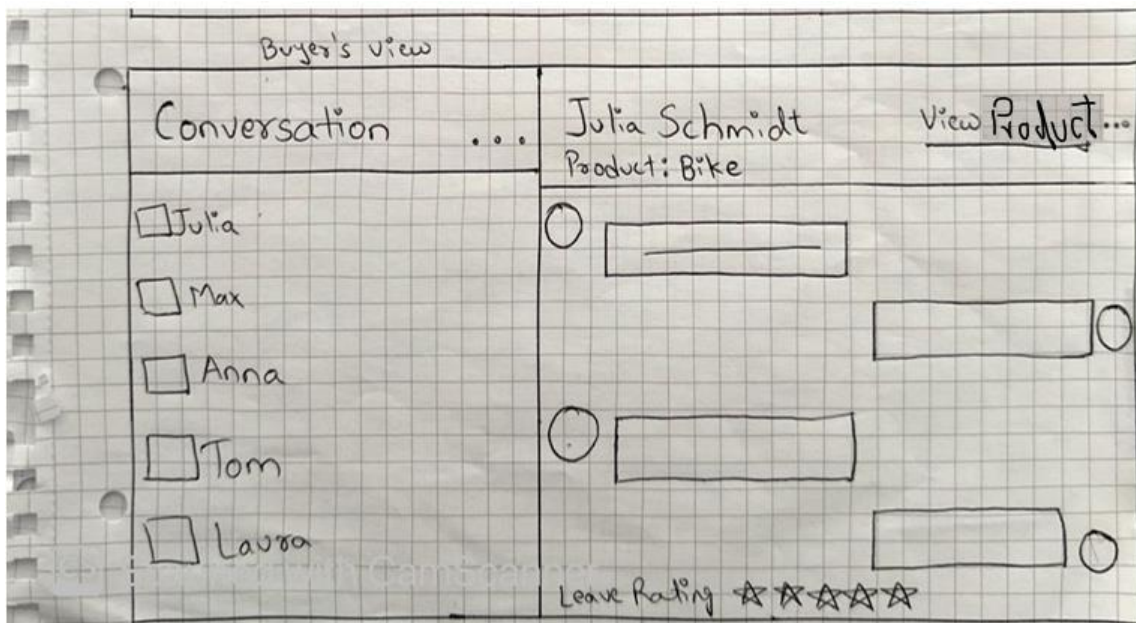
Description:

This page provides registered users with a personal dashboard to manage all their listings. It helps users keep track of the status of each product they have posted, whether it's active, pending approval, hidden, sold, or expired. Sellers can take quick actions like editing or deleting listings and monitor user engagement.

Key Features:

- **Search Bar:** Easily locate specific listings.
- **Status Tabs:** Filter listings based on status (Active, Pending, Hidden, Sold, Expired).
- **Product Cards:** Display product details along with options to edit, hide, or delete the listing.

4.9 Messaging Interface



Description:

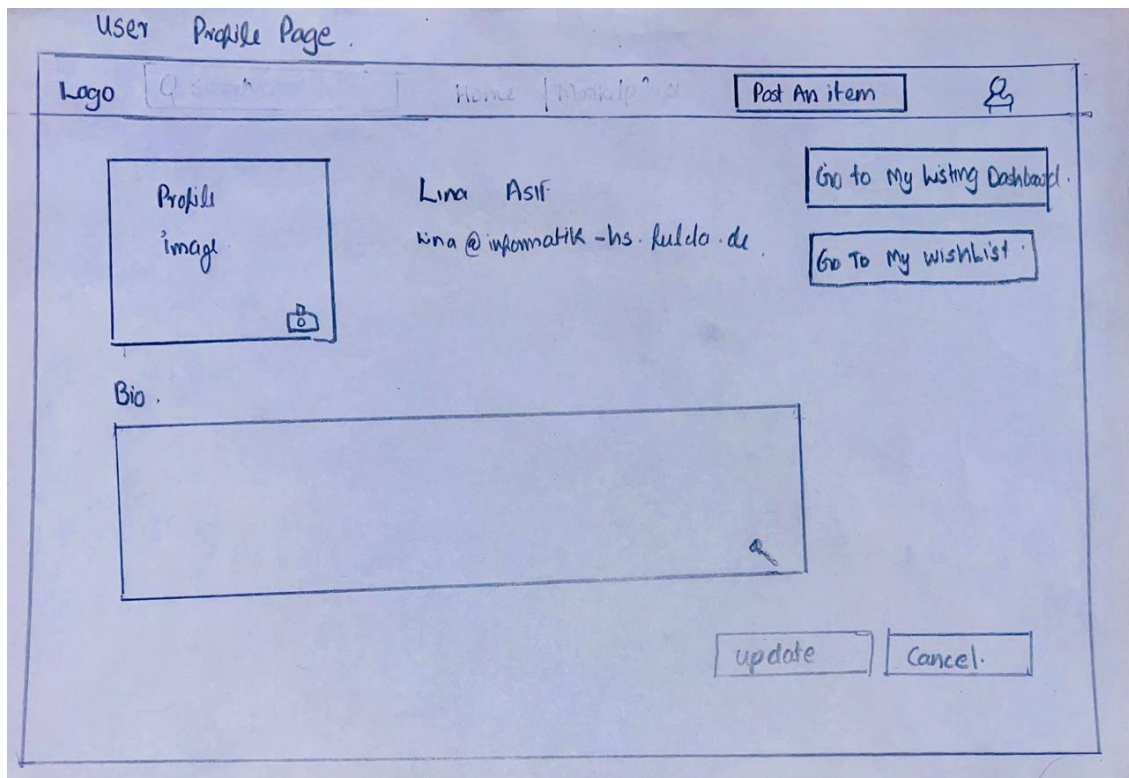
The messaging system allows buyers and sellers to interact with each other. Sellers can view a list of ongoing conversations, reply to messages, and take actions such as reporting users or marking conversations as unread. This feature ensures secure and organized communication within the platform.

Key Features:

- **Conversation List:** Sidebar with active message threads.
- **Message Panel:** Displays ongoing chat with buyers.
- **Action Menu:** Options to mark messages, report users, delete conversations, or finalize sales directly from the chat window.

- **Reply Box:** Easy-to-use input field for responding to messages.

4.10 User Profile



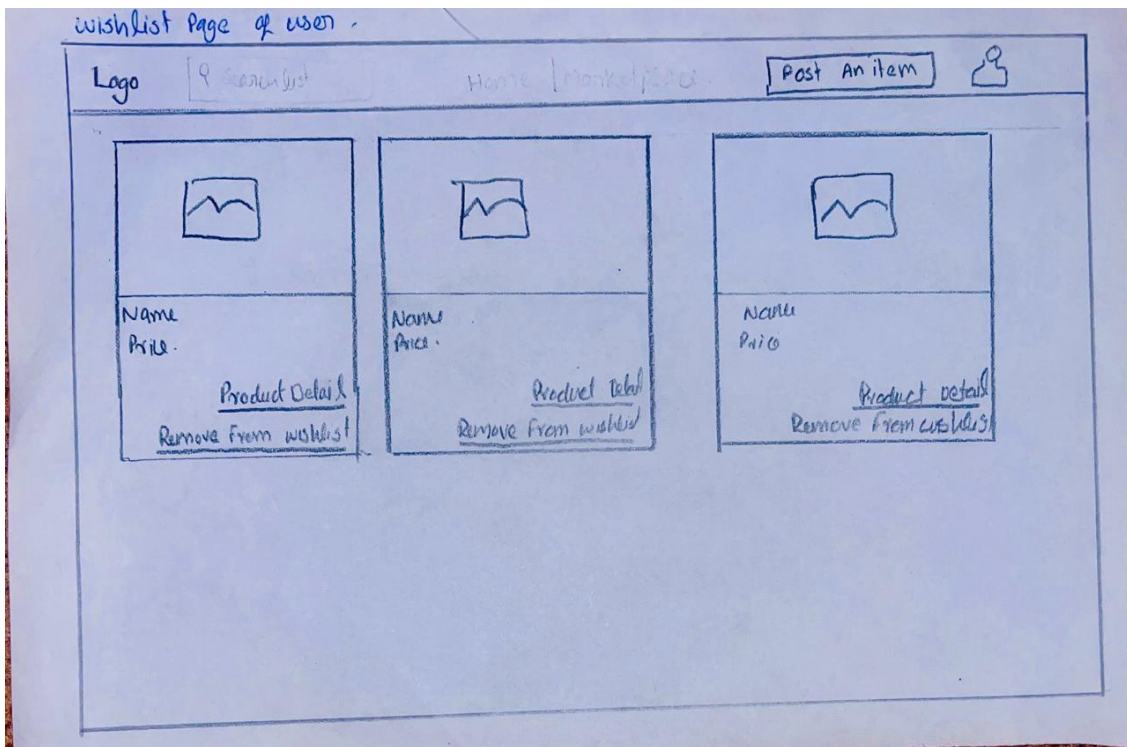
Description:

This user profile page displays personal information and allows users to manage their account details. It provides a space for users to present themselves to the marketplace community with customizable bio information.

Key Features:

- **Update Profile Picture:** Profile image upload and display functionality
- **Update Personal Information:** User name and email information display

4.11 Wishlist viewing page



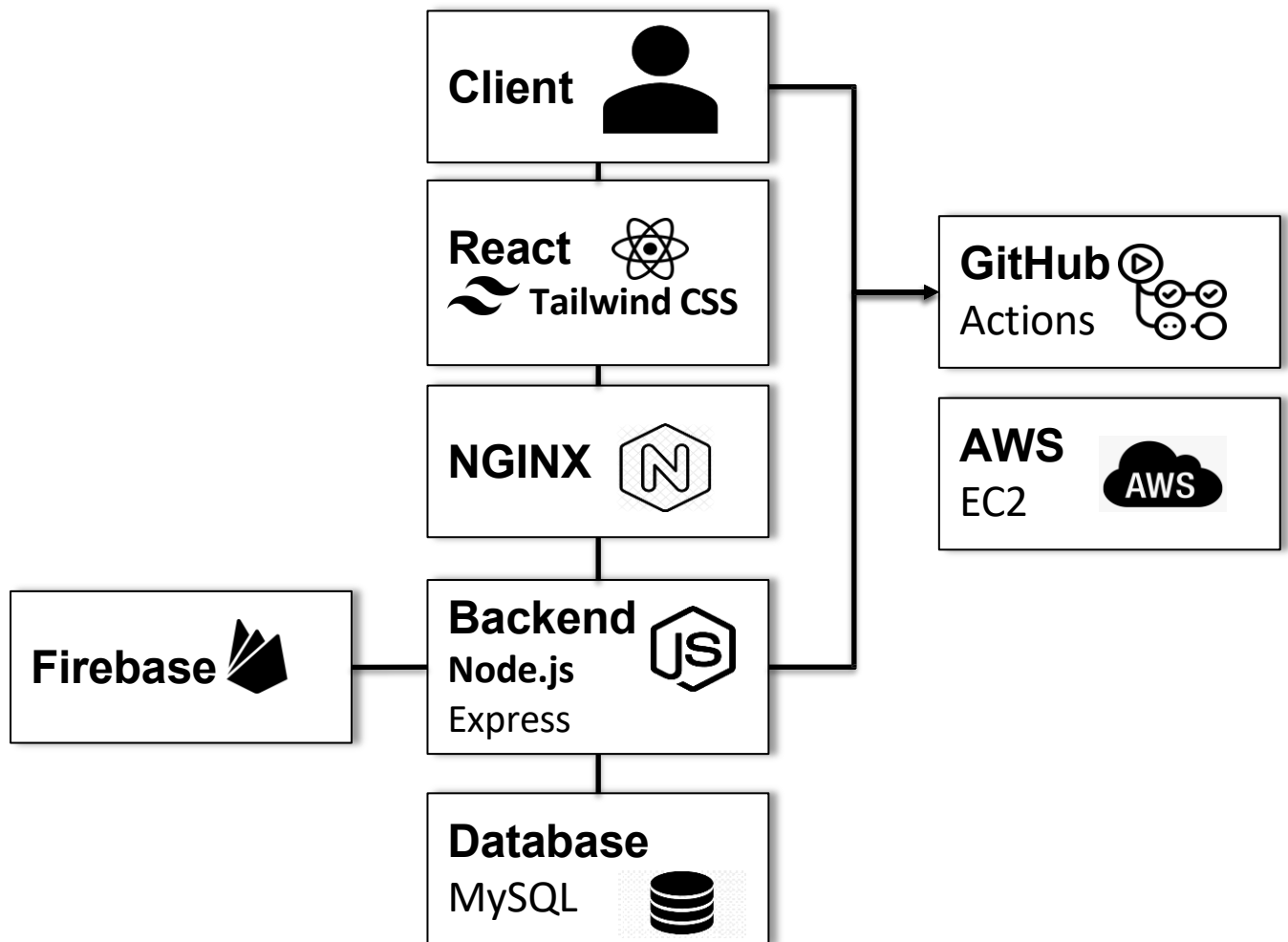
Description:

This wishlist page display all the products wishlisted by the user . This can be used by the users to view their favorite items later.

Key Features:

- **Remove from wishlist:** Users can remove the wishlisted product if its not needed anymore.

5. NEW HIGH LEVEL ARCHITECTURE, DATABASE ORGANIZATION



5.1 DB ORGANIZATION

Our application uses a relational database (MySQL 8.0) deployed on AWS EC2. The main tables are:

- **Users:** Stores registered users (buyers, sellers, moderators)
- **ProductListings:** Contains items for sale, their metadata, and creator
- **MediaFiles:** Holds file paths for uploaded images/videos

- Chat: Links user pairs to specific product listings with message context
- Messages: Stores all in-app communications between users
- Favorites: Tracks which products are saved by users
- Reports: Allows users to report inappropriate listings or users
- Search History: Logs filter parameters used by users

5.2 MEDIA STORAGE

Media (images) will be stored externally using Firebase Cloud Storage, not in the database as BLOBs. Only the image URL will be stored in the database.

5.3 SEARCH/FILTER ARCHITECTURE AND IMPLEMENTATION

In backend, we will use SQL LIKE to search the user input from product title, description, and tags. After that, we will use SQL WHERE to filter the results. The user can filter by category, price range, condition (new or used), and location. These filters will help users to find the product more easily. Sorting will be done on server side using ORDER BY clause, for example by date or price.

5.4 APIS

Listings API

- GET /api/listings – Retrieve a list of all product listings
- POST /api/listings – Create a new product listing
- PUT /api/listings/{id} – Update a listing by ID
- GET /api/listings/{id} – Retrieve a specific product listing
- DELETE /api/listings/{id} – Delete a listing by ID
- GET /api/listings/search – Search for listings by keyword

Users API

- POST /api/users – Register a new user
- GET /api/users – Get all users
- GET /api/users/{id} – Get user by ID
- PUT /api/users/{id} – Update user profile info (name, picture, etc.)

Favourites API

- POST /api/favourites – Add a product to favourites
- GET /api/favourites/{userId} – Get a user's favourites
- DELETE /api/favourites/{productId} – Remove product from favourites

Reports API

- POST /api/reports – File a report
- GET /api/reports/{id} – Get details of a specific report
- GET /api/reports/user/{userId} – Get all reports of a specific user
- GET /api/reports – Retrieve all reports

Chat API

- GET /api/chat/user/{userId} – Get all chats for a user (includes receiver name, product image, last message, etc.)

Messages API

- GET /api/messages/chat/{chatId} – Retrieve all messages for a specific chat
- POST /api/messages – Send a message (stores sender_id, receiver_id, product_id, content)

5.5 NON-TRIVIAL ALGORITHM

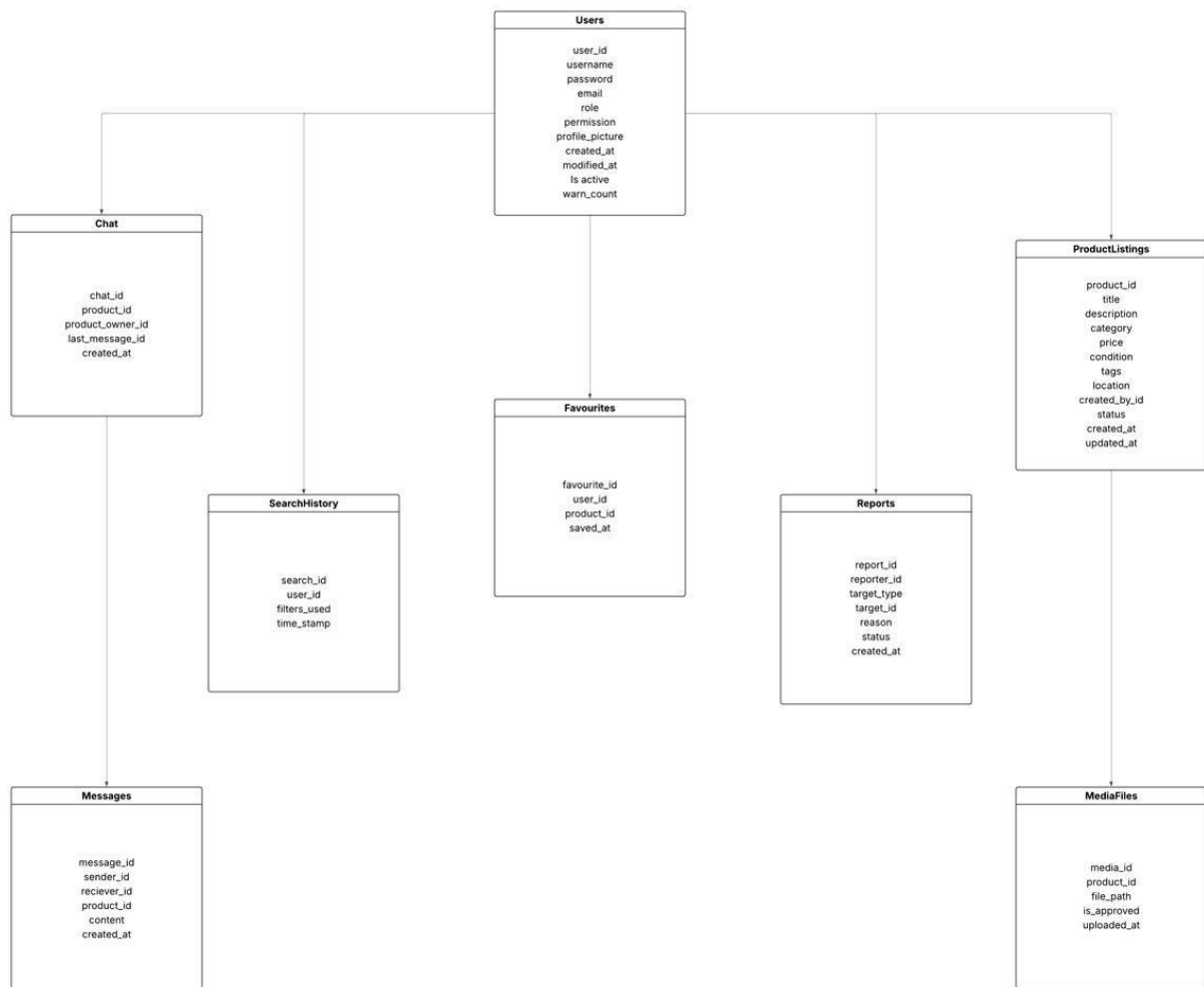
Our application does not implement any non-trivial or custom algorithms. All logic is handled through standard CRUD operations and SQL queries.

5.6 TOOLS AND FRAMEWORKS

We continue using the tools and frameworks listed in Milestone 1. There is no modification in them.

6. HIGH LEVEL UML DIAGRAMS

This section will contain UML diagrams describing the system architecture, class relations, and other modeling views.



7. IDENTIFY ACTUAL KEY RISKS FOR YOUR PROJECT AT THIS TIME

7.1 SKILL RISK

7.1.1 Lack of skill

Risk: We have 2 team members for frontend and 3 for backend and 1 GitHub Master. Few team members don't have any experience in developing real time applications and they are learning new technologies and there is a risk involved that incase those skills of a certain team member are not sufficient to build the project.

Solution: Pair less experienced developers with more knowledgeable team members for mentorship. Allocate initial project time (e.g., first week) for crash-course tutorials, prototyping, and tech exploration.

7.2 SCHEDULE RISKS

7.2.1 Time Slots Not Overlapping

Risk: Team members have different schedules due to other projects, classes, and jobs, which risks disrupting communication and project progress.

Solution: Meetings will be planned in advance to accommodate everyone's schedules and ensure full participation.

7.2.2 Team Member's Ability to Work

Risk: A team member might suddenly be unable to work, for example, due to health issues.

Solution: By planning ahead, work can be reassigned to other team members who can take over the affected workload.

7.2.3 Overcommitment of Team Members

Risk: Due to team members working on multiple projects or studying for exams, there might be weeks when some members can barely contribute to this project, potentially causing delays.

Solution: The team will communicate weekly about each member's available time and distribute tasks accordingly. Lost time will be compensated by increased effort in the following weeks.

7.3 TEAMWORK RISKS

7.3.1 Miscommunication and Conflicts Within the Group

Risk: During the project, conflicts or miscommunication within the group may occur.

Solution: To address such issues, a discussion round will be established where problems can be resolved collaboratively by team members.

7.3.2 Uneven Contribution

Risk: Some members may contribute significantly less, leading to frustration and imbalance in work distribution.

Solution: Set transparent weekly goals for each member. Log contributions through version control and task tracking tools.

7.4 Technical Risks

7.4.1 Exceeding AWS Student Credits

Risk : The project is being developed using AWS student credits These credits have a limited balance, and if the usage exceeds this limit, the team may incur unexpected charges or be forced to halt cloud-based development.

Solution: Regularly monitor credit usage through the AWS Billing & Cost Management Dashboard. Keep EC2 instances stopped when not actively being used and delete unused volumes or snapshots to reduce storage costs.

8. PROJECT MANAGEMENT

Our workflow for M2 Document tasks was as follows:

- On Friday after meeting, we discuss what needs to be done.
- Each member gets assigned tasks (assigned in Trello to track) and works on them until the team meeting on Wednesday.
- On Wednesday, we check the work done together and discuss what to change until the Friday meeting.

The workflow for the M2 Vertical SW Prototype and future tasks will be:

- At the end of the Friday meeting, discuss and assign tasks.
- The team lead updates Trello accordingly, so team members can check what work they were assigned.
- On Wednesday the team meets, reviews the completed work, discusses it, and decides on changes before Friday.
- If any problem occurs, the team will meet online or offline to find a solution.
- For communication, the team uses WhatsApp and Google Meet.

Swoplet

Milestone 4: Next Beta Launch, QA and Usability Testing and Final Commitment for Product Features (P1 list)

Team 6

Team Lead Email: hammad.asif@informatik.hs-fulda.de

Team Member	Role
Hammad Asif	Team Lead
Karan Patel	Git master
Pooja Puthu Vayal	Backend Lead
Akhil Sajan	Frontend Lead
Hamza Butt	Frontend Developer
Hasara Koralege	Backend Developer

History

Date	Version
11/07/2025	First Draft

1. Product summary

Purchasing, selling, and exchanging products within the campus community can provide challenges for university staff, faculty, and students. Their particular needs are usually not met by public platforms, raising questions about security, pricing fairness, and item authenticity.

Swoplet is a safe online platform created just for the Fulda University community to close this gap. It guarantees that the platform can only be accessed and used by confirmed university members to safely purchase, sell, or exchange goods and services. With a university-verified user base, moderator oversight for increased security, and an AI-powered pricing assistant that assists users in estimating fair market prices prior to transactions, Swoplet provides a smooth user experience that is suited to the demands of campus commerce. This encourages transparency and empowers users to make well-informed decisions.

Final Committed Major P1 Functions

- Users must register with their official university email address.
- Users can browse available listings on the platform.
- Users can search for specific listings using keywords.
- Users can filter and sort listings by category, price, or time posted.
- Buyers can securely message sellers through an internal messaging system.
- Buyers can save items to a wishlist for future reference.
- Sellers can create new listings with item details like title, price, and description.
- Sellers can upload images of their product.
- Sellers can respond to buyer inquiries using the internal messaging system.
- Sellers have a dashboard to manage listings (edit, delete, mark sold, etc.).
- Sellers can temporarily hide a listing from public view.
- Sellers must categorize listings properly and add relevant tags.
- Admin must approve listings before they are visible to others.
- Admin can moderate the platform by removing content or banning users.

Unique Selling Point:

Includes an AI-based pricing suggestion feature that recommends fair market prices based on current listing trends, aiding both buyers and sellers in making informed decisions.

Deployment URL:

[Swoplet](#)

2. Usability test plan

2.1. Test objectives

- **Effectiveness:**
Can sellers complete the full process of posting a listing with all required fields, images, and metadata without any help?
- **Efficiency:**
How long does it take users to go from the **dashboard** to a **successfully published listing**?
Where do they get stuck?
- **Satisfaction:**
How do users rate their overall experience in terms of usability, communication, reliability, and performance?

2.2. Test Background & Setup

System Under Test:

[Swoplet Live Site](#)

- **Test Environment:**
 - Devices: Laptop, desktop, mobile phones
 - Browsers: Chrome (latest), Edge (latest), Safari (mobile)
 - Network: Stable high-speed Internet
- **Test Participants:**
 - 5–10 real users or representative testers (preferably unfamiliar with the platform)

2.3. Test Roles

Role	Responsibility
Facilitator	Gives instructions, observes behavior, avoids interfering
Note-taker	Logs issues, confusion points, time metrics
User/Test Subject	Completes tasks while thinking aloud

2.4. Test Tasks & Instructions

1. Login

- a. Go to the Swoplet homepage and log in using the provided seller's test credentials or create a new user.

2. Navigate

- a. From the dashboard, locate and click the **“Post New Item”** button.

3. Create Listing

- a. Fill in product listing details:
 - i. Realistic **title, description**
 - ii. Select appropriate **category**
 - iii. Add **price, availability**
 - iv. Upload **minimum 2 images**
 - v. Add relevant
- b. Click **“Create Listing”** to complete.

4. Verify Listing

- a. Go to **“My Listings”** and confirm that the new item appears correctly.

5. Post-Task

- o Complete the **Satisfaction Questionnaire** (see 2.1.6).

2.5. Metrics & Data Collection

Metric	Description
Success Rate	% of users who publish a listing without errors
Time on Task	Time taken from dashboard to published listing
Error Frequency	Upload failures, field errors, navigation misclicks
User Comments	Verbal or written feedback

Observations	Points of hesitation, repeated attempts, UI confusion
---------------------	---

2.6. Post-Test Satisfaction Questionnaire

All questions use a 1–5 Likert scale (1 = Strongly Disagree, 5 = Strongly Agree)

Ease of Use

- Swoplet is easy to navigate and use.
- Creating and managing product listings is straightforward.
- Finding products, I want to buy is efficient.

Communication

- The chat system makes communication with other users easy.
- I feel comfortable and safe when interacting with other users.

Trust & Reliability

- The seller rating system helps me make informed decisions.
- The platform feels secure and trustworthy.

Performance

- The website loads quickly and works smoothly.
- I rarely encounter errors or technical issues.

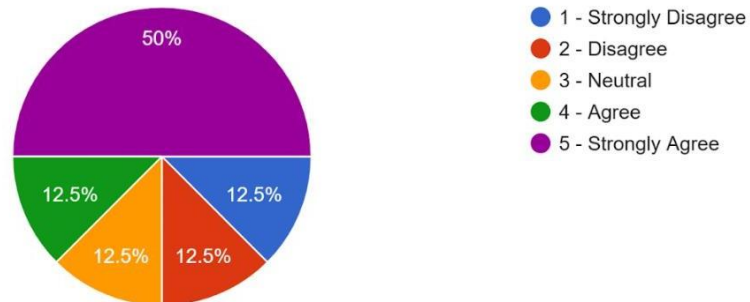
Overall Satisfaction

- I am satisfied with my overall Swoplet experience.
- I would recommend Swoplet to others.
- I plan to continue using Swoplet in the future.

2.7. Feedback Analysis

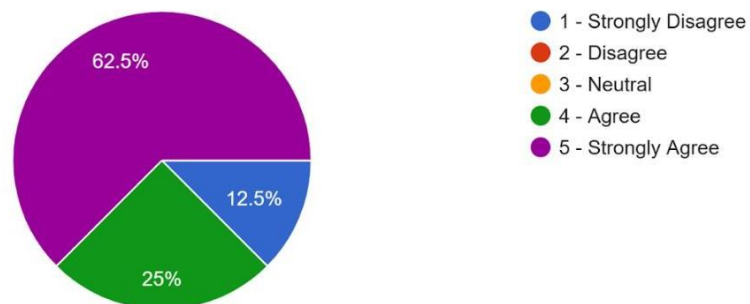
Swoplet is easy to navigate and use.

8 responses



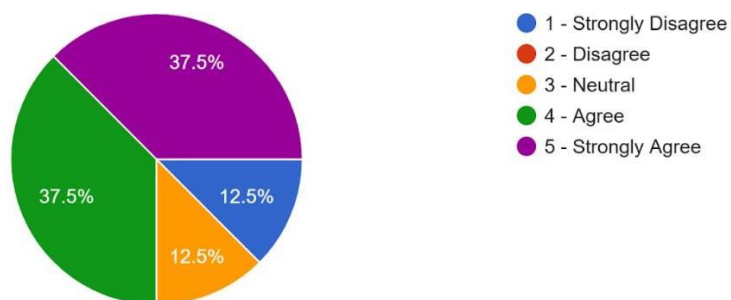
Creating and managing product listings is straightforward.

8 responses



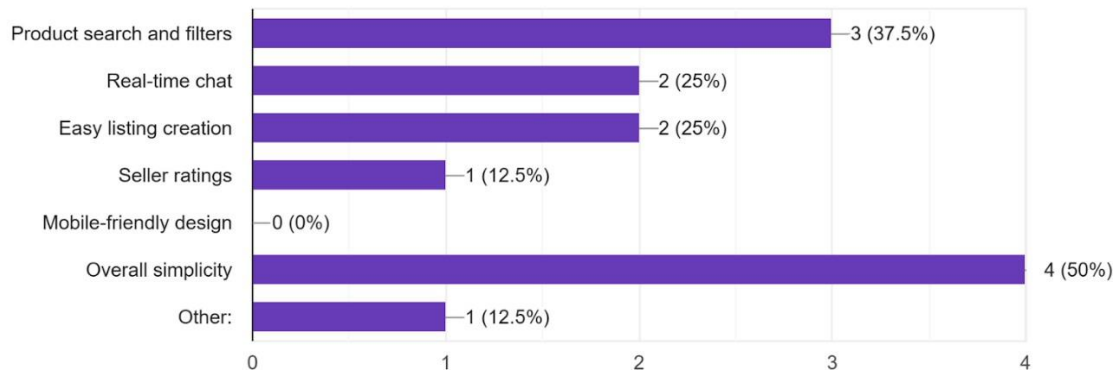
I am satisfied with my overall Swoplet experience.

8 responses



What's your favorite feature of Swoplet?

8 responses



3. QA test plan

3.1. Test Objectives

The primary objective of this QA test plan is to ensure the functionality and reliability of the "Create Product Listing" feature. To ensure that users can successfully create a new product listing with required fields (title, description, price, tags, images, etc.), and that form validations, file uploads, and API responses work as expected across different browsers.

3.2. HW and SW setup

Component	Details
OS	Windows 10 / macOS Ventura
Browsers	Google Chrome (v124+) and Microsoft Edge (v124+)
Device	Laptop/Desktop
URL	swoplet-zler.onrender.com
Backend API	https://swoplet-zler.onrender.com/api/
Database	swoplet-db.cx4q28o46i78.eu-central-1.rds.amazonaws.com (Amazon RDS)

3.3. Feature to be tested

- Product creation form on the "Create Listing" page
- Field validations (required, format)
- Media file upload and preview
- API submission and response
- UI feedback after successful listing

3.4. QA Test plan

Test #	Test Title	Test Description	Test Input	Expected Output	Chrome Result	Edge Result
1	Create Listing – Success	Submit all required fields to create listing successfully	Title: "Hair Dryer" Description: "HH Simonsen Compact Dryer, black. Immediately available, delivery time: 1-3 days" Price: \$99 Category: Appliances Condition: Used-Good Tags: "hair dryer, used" Image: JPEG	After creating the listing, a notification appears in the UI stating that the product was successfully created. The user stays on the same page, and the form is reset.	PASS	PASS
2	Create Listing Success & My Listings	Create listing successfully and check user's listing page	Same input as Test 1	The new product listing appears on the user's My Listings page under the Pending category.	PASS	PASS
3	Create Listing Success & Admin Dashboard	Create listing successfully and check admin dashboard	Same input as Test 1	The new product listing appears on the Admin Dashboard page under the Pending list.	PASS	PASS
4	Missing Required Fields	Submit form without title or price	Title: "" Price: "" Other fields filled	Validation error messages are displayed for	PASS	PASS

Test #	Test Title	Test Description	Test Input	Expected Output	Chrome Result	Edge Result
				required fields; form is not submitted.		
5	Image Upload Validation	Upload a file with an unsupported format	Upload: document.pdf	Error message displayed: "Only image files are allowed."	PASS	PASS
6	Large Image Upload	Try uploading an image larger than 5MB	Upload: image_6mb.jpg	Error displayed: "File size exceeds limit," or upload fails gracefully.	PASS	PASS
7	Create Without Category	Submit listing without selecting a category	Leave category field empty	Validation error displayed: "Please select a category."	PASS	PASS

4. Code Review

4.1. Coding Style

- Consistent use of whitespaces (for example, nested code should be tabbed in)
- Names for functions, classes, variables, etc. need to clearly describe what they are used for/what they do
- Naming is done in camelCase (example: "getListingData()")
- DRY (Don't Repeat Yourself) principle: No duplicate code; if the same code is needed multiple times, it will be moved into a function-call
- KISS (Keep It Simple, Stupid) principle: Code is only as complex as it needs to be
- Code that isn't straightforward has comments that describe what it does
- Proper Error Handling is done (example: try-catch)
- A clean file order is used (example: controller calls service calls repository)

4.2. Review: AdminDashboard.jsx

Code writer: Akhil Sajan

Reviewer: Pooja Pv

Date: 27.06.2025

File to be reviewed: main/UI/src/Pages/AdminDashboard/AdminDashboard.jsx

Overview

The AdminDashboard component is part of an admin panel interface allowing administrators to manage product listings and monitor reported users. It fetches listings based on their approval status (pending, approved, rejected) and allows admins to approve/reject pending listings. It also provides a tab to view reported users.

Strengths and Positive Aspects

- **Modular & Clean Structure:** The code is split into logical sections: state management, side effects (useEffect), and handlers.
- **Reusability with useApiRequest Hook:** API calls are abstracted into a reusable custom hook, improving testability and reducing repetition.
- **Role-Based Access Control:** Includes a check to restrict access based on admin email.
- **Responsive Design:** Utilizes Tailwind CSS for a responsive, visually appealing layout.
- **Clear UI Logic:** Conditionals based on activeTab make the user interface intuitive and manageable.

Areas of Improvement

1. The same fetchProducts function is called multiple times with only the status value changing. This violates DRY (Don't Repeat Yourself) principles and can be simplified using a mapping object.

```
33     useEffect(() => {
34         if (activeTab === "pending") {
35             fetchProducts({
36                 url: `${BASE_URL}products/search`,
37                 method: "POST",
38                 body: { status: 0 },
39                 headers: { "Content-Type": "application/json" },
40             });
41         } else if (activeTab === "approved") {
42             fetchProducts({
43                 url: `${BASE_URL}products/search`,
44                 method: "POST",
45                 body: { status: 1 },
46                 headers: { "Content-Type": "application/json" },
47             });
48         } else if (activeTab === "rejected") {
49             fetchProducts({
50                 url: `${BASE_URL}products/search`,
51                 method: "POST",
52                 body: { status: 2 },
53                 headers: { "Content-Type": "application/json" },
```

2. This product rendering block is copied three times. Consider creating a small `renderProducts(products, status)` function or a reusable component to avoid repeating the same JSX. It's easier to maintain if the design changes later.

```
293         {productsData.products
294           .filter((product) => product.status === 2)
295           .map((product) => (
296             <AdminProductCard
297               key={product.id}
298               title={product.title}
299               image={
300                 product.media_link ||
301                 product.image_url ||
302                 "https://via.placeholder.com/150"
303             )
304           )
305         }
```

3. Optional improvement: Consider putting "admin@hs-fulda.de" into a config file or constant, so it's easier to maintain.
4. You can remove unused items like `searchQuery`, the `<ProductCard />` at the end, and any other variables not being used to keep the code clean. Also, adding a few short comments above key functions like `handleApprove` and `handleReject` would improve readability.

4.3. Review: product listing controller

- **Code writer:** Pooja
- **Reviewer:** Akhil Sajan
- **Date:** 27.06.2025
- **File to be reviewed:** Backend/controllers/productlisting.controller.js

Overview

This controller manages core product listing operations:

- Fetching listings by status (approved, by ID, by user, etc.)
- Creating, updating, and soft-deleting listings
- Updating product status (including marking products as sold)
- Enriching listings with related data (eg, seller review stats, media files, categories)

It makes good use of Sequelize's ORM features and keeps business logic mostly inside controller functions.

Strengths and Positive Aspects

- **Separation of concerns:** Controller logic is organized cleanly, and data access/manipulation is handled by Sequelize models.
- **Error handling:** Each function is wrapped in try/catch blocks and provides clear, meaningful error messages to the client.
- **Reusability:** The helper function `getSellerReviewStats` abstracts seller rating calculation, preventing repeated aggregation logic across endpoints.
- **Soft deletion:** Uses a `is_deleted` flag to mark records instead of permanently deleting them, which supports safer data recovery.
- **Data enrichment:** Listing endpoints include related entities like media files, categories, conditions, creators, and seller review stats, improving the quality of response data.
- **Bulk operations:** Efficiently handles media replacement with `bulkCreate` and cleans up dependent data (like wishlists) during updates.
- **Clear status management:** Implements careful status tracking by resetting status to pending on updates and preserving the previous status for historical context.

Areas of Improvement

- **Code repetition:** Many `findAll` and `findByPk` calls repeat similar include clauses. Extract common includes into a constant (eg, `const PRODUCT_INCLUDES = [...]`) to DRY up the code.

```
const listings = await productlisting.findAll({
  attributes: [
    'id', 'title', 'description', 'price', 'tags', 'location', 'status', 'created_at', 'updated_at'
  ],
  where: {
    status: APPROVAL_STATUS.APPROVED,
    is_deleted: DELETE_STATUS.ACTIVE
  },
  include: [
    { model: mediafile, as: 'mediafiles', attributes: ['file_path'], where: { is_approved: true }, required: true },
    { model: productcategory, as: 'category' },
    { model: productcondition, as: 'condition' },
    { model: user, as: 'creator' }
  ]
});
```

- **Hardcoded status values:** Status values like 3(sold) and 1(approved) are used directly; replace them with named constants from `APPROVAL_STATUS` to improve readability and reduce errors.

```
await product.update({
  previous_status: product.status,
  status: 3, // 3 = sold
  updated_at: new Date().toISOString()
});
```


4.4. Review: ProductDetailPage.jsx

Reviewer: Hasara Koralege

Code writer: Hammad Asif

Date: 27.06.2025

File to be reviewed: UI\src\Pages\ProductDetail\ProductDetailPage.jsx

Overview

The ProductDetailPage.jsx file defines the product detail page, which displays comprehensive information about a selected product. It fetches product data from the backend using the product ID from the URL, and renders details such as images, description, seller information, and reviews. The page also allows users to add/remove the product from their favourites and contact the seller, with appropriate UI feedback and backend calls. Tabs are used to organize product information, and toast notifications provide user feedback for actions.

Strengths and Positive Aspects

1. The code is modular and leverages React hooks effectively for state and side effects.
2. API calls are abstracted using a custom hook (useApiRequest), improving reusability and separation of concerns.
3. The UI is responsive and well-structured, with clear separation between product details, images, and actions.

Areas of Improvement

1. Remove unwanted imports to keep code clean. You import SearchBar, but don't use it anywhere.



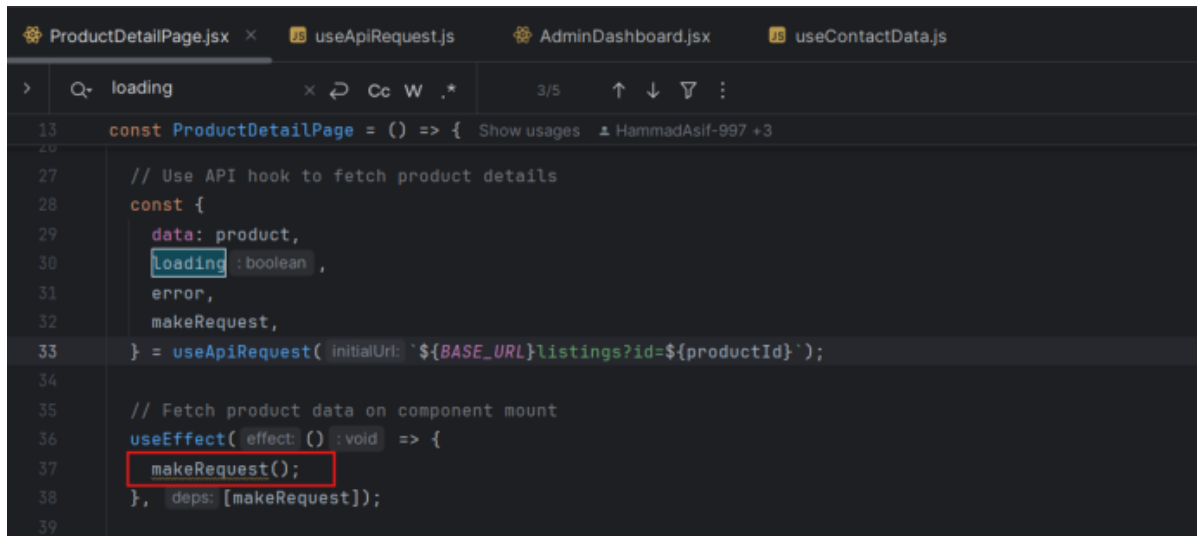
```
ProductDetailPage.jsx x
1  import { ChevronLeft, ChevronRight } from "lucide-react";
2  import SearchBar from "../../components/Common/SearchBar";
3  import React, { useState, useEffect } from "react";
4  import { useApiRequest } from "../../hooks/useApiRequest";
5  import { BASE_URL, userId } from "../../constants/config";
6  import LoadingState from "../../components/Common/LoadingState";
7  import ErrorState from "../../components/Common/ErrorState";
```

2. Avoid native alert() in modern UIs. alert() blocks the main thread, interrupts the user experience, and looks outdated. Instead add userfriendly non-blocking notification of error in configured position.

```
ProductDetailPage.jsx x
13  const ProductDetailPage = () => { Show usages  HammadAsif-997 +3
164  const handleContactSeller = async () : Promise<void> => { Show usages  HammadAsif-997
165      // If not logged in, redirect to login
166      if (!currentUserId) {
167          navigate("/login");
168          return;
169      }
170
171      // Don't allow contacting yourself
172      if (parseInt(currentUserId) === product.created_by_id) {
173          alert("You cannot contact yourself!");
174          return;
175      }
176
177      setIsContactingSeller( value: true);
```

```
ProductDetailPage.jsx x
13  const ProductDetailPage = () => { Show usages  HammadAsif-997 +3
164  const handleContactSeller = async () : Promise<void> => { Show usages  HammadAsif-997
186
187      // Store information about the chat we want to open
188      localStorage.setItem('activeChatInfo', JSON.stringify( value: {
189          sellerId: product.created_by_id,
190          productId: product.id,
191          sellerName: product.creator.username,
192          productTitle: product.title
193      }));
194
195      // Navigate to chat page regardless of whether chat existed or was newly created
196      navigate("/messages");
197      } catch (error) {
198          console.error("Error contacting seller:", error);
199          alert("Failed to contact seller. Please try again.");
200      } finally {
201          setIsContactingSeller( value: false);
202      }
203  };
204
```

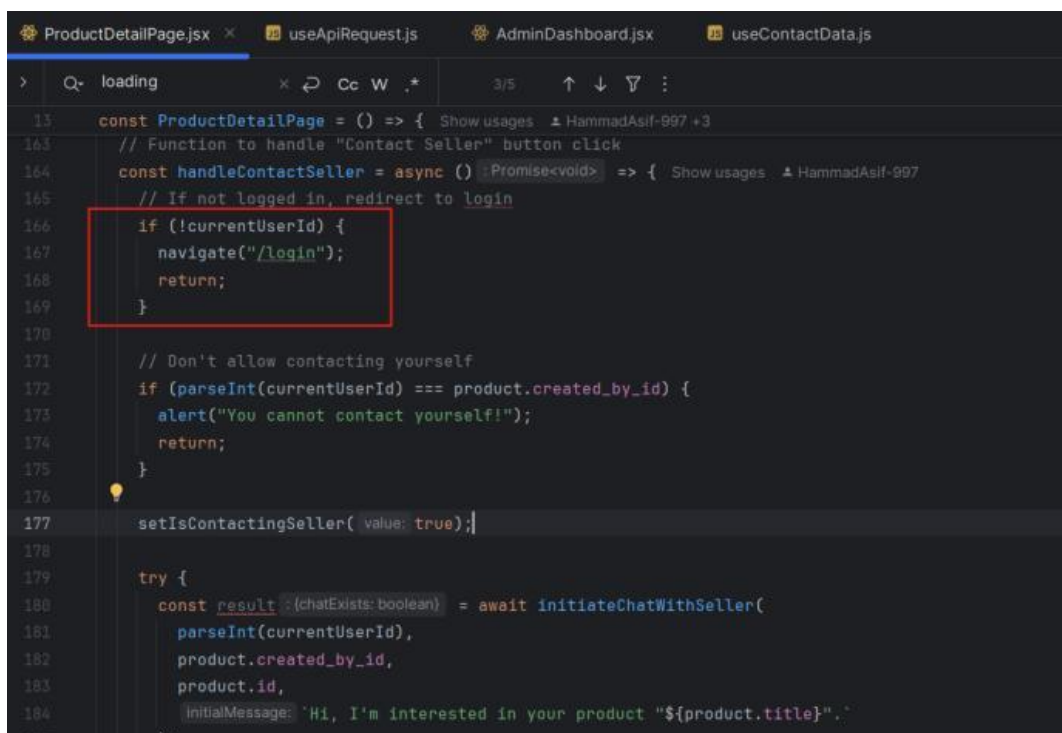
3. I recommend explicitly passing the url as an override when calling makeRequest. While useApiRequest takes initial Url on initialization, it won't update initialUrl if productId changes later without remounting the component. By explicitly providing the url to makeRequest(), we ensure the latest productId is always used for the API request, even if the component stays mounted. This makes the data fetching more robust if productId can change dynamically



The screenshot shows a VS Code editor with four tabs: ProductDetailPage.jsx, useApiRequest.js, AdminDashboard.jsx, and useContactData.js. The ProductDetailPage.jsx file is open, showing a search bar with 'loading' and a line number indicator from 13 to 39. The code defines a `ProductDetailPage` component using `useApiRequest` and `useEffect`. A red box highlights the `makeRequest()` call inside the `useEffect` dependency array.

```
13 const ProductDetailPage = () => { Show usages HammadAsif-997 +3
20
27 // Use API hook to fetch product details
28 const {
29   data: product,
30   loading: boolean,
31   error,
32   makeRequest,
33 } = useApiRequest( {initialUrl: `${BASE_URL}listings?id=${productId}`});
34
35 // Fetch product data on component mount
36 useEffect( effect: () :void => {
37   makeRequest();
38 }, deps: [makeRequest]);
39
```

4. Extract a reusable function for the auth check + redirect and define a helper function inside your component or in a utils file.



The screenshot shows the same VS Code editor with the ProductDetailPage.jsx file. The code now includes a `handleContactSeller` function. A red box highlights the `if (!currentUserId)` block, which calls `navigate("/login")` and returns. Below this, there is a check to prevent contacting oneself and a call to `setIsContactingSeller`. The `try` block contains a call to `initiateChatWithSeller`.

```
163 // Function to handle "Contact Seller" button click
164 const handleContactSeller = async () :Promise<void> => { Show usages HammadAsif-997
165   // If not logged in, redirect to login
166   if (!currentUserId) {
167     navigate("/login");
168     return;
169   }
170
171   // Don't allow contacting yourself
172   if (parseInt(currentUserId) === product.created_by_id) {
173     alert("You cannot contact yourself!");
174     return;
175   }
176
177   setIsContactingSeller( value: true);
178
179   try {
180     const result :{chatExists: boolean} = await initiateChatWithSeller(
181       parseInt(currentUserId),
182       product.created_by_id,
183       product.id,
184       {initialMessage: `Hi, I'm interested in your product "${product.title}"`},
185     );
186   } catch {
187     // Handle error
188   }
189 }
```

```
13 const ProductDetailPage = () => { Show usages HammadAsif-997 +3
112 // Function to handle "Add to Favourite" or "Remove from Favourite" button click
113 const handleFavouriteToggle = async () : Promise<void> => { Show usages HammadAsif-99
114 // If not logged in, redirect to login
115 if (!currentUserId) {
116   navigate("/login");
117   return;
118 }
119
120 setIsAddingToFavourite( value: true);
121
122 try {
123   if (isProductInFavourite) {
124     // Remove from favourites
125     const response : Response = await fetch(
126       input: `${BASE_URL}favourites?user_id=${currentUserId}&product_id=${product
127       init: { method: "DELETE" }
128     );
```

4.5. Review: *sellerreview.controller.js*

- **Code writer:** *Hasara Koralege*
- **Reviewer:** Hammad Asif
- **Date:** 27.06.2025
- **File to be reviewed:** Backend/controllers/sellerreview.controller.js

Overview

This file [seller-review.controller.js](#) provides two main controller methods:

- **getPendingSellerReviews:** Fetches sellers who a given buyer is yet to review, along with product and seller data.
- **addSellerReview:** Saves a new review for a seller, marks the corresponding seller_to_review entry as reviewed, and stores the rating and optional comment.

The module ensures buyers are reminded to review sellers after transactions and supports a transparent review system with average rating tracking.

Strengths and Positive Aspects

- Code is clean, readable, and logically structured.
- Average rating and total count are dynamically calculated with helper function `getSellerReviewStats()`.
- Effective use of Sequelize's include and conditional querying.

- Error handling is properly implemented using try/catch.
- Clear response messages and appropriate HTTP status codes are used.
- Makes good use of optional chaining (e.g., `entry.product?.mediafiles[0]?.file_path`).
- Handles nullable values like `review_comment` gracefully.

Areas for Improvement

- **Performance Optimization:** The use of `Promise.all()` for each seller review entry might create N additional DB calls for `getSellerReviewStats()`. This could be optimized with a batch query or caching mechanism.
- **Commenting:** Some logic blocks (especially within `map`) would benefit from brief comments.
- **Validation Layer:** Input validation should ideally be extracted into middleware (e.g., using `express-validator`) for cleaner controller code.
- **File Organization:** Moving `getSellerReviewStats()` to a service layer would make the controller slimmer and more testable.
- **Typo Risk:** Consistency in object keys could be improved (`reviewee_id` vs. `reviewer_id` — confirm naming aligns across models).

Specific Code Comments

Figure	LINE	COMMENT
1	5-14	Good use of helper function to centralize average rating logic. Could be moved to a service.
2	23-25	Input validation could be moved to middleware.
3	29-41	Efficient data fetching using Sequelize <code>include</code> with aliasing. Good use of <code>required: false</code> .
4	43-54	<code>Promise.all()</code> with nested DB calls could impact performance for large datasets.
5	92	Considers optional <code>review_comment</code> . Nice defensive programming.

```

3 // Helper function to get seller's total reviews and average rating
4 async function getSellerReviewStats(seller_id) {
5   const reviews = await sellerReview.findAll({
6     include: [{
7       model: sellerToReview,
8       as: 'toReview',
9       where: { seller_id },
10      attributes: []
11    }],
12    attributes: ['review_value']
13  });
14  const total = reviews.length;
15  const sum = reviews.reduce((acc, r) => acc + (r.review_value || 0), 0);
16  const avg = total > 0 ? sum / total : 0;
17  return { total, avg };
18 }

```

Figure 1: seller-review.controller.js Code 1

```

20 // GET /seller-to-review?buyer_id=xxx
21 exports.getPendingSellerReviews = async (req, res) => {
22   try {
23     const { buyer_id } = req.query;
24     if (!buyer_id) {
25       return res.error('buyer_id is required', 400);
26     }
27   }

```

Figure 2: seller-review.controller.js Code 2

```

28 // Find all seller_to_review entries for this buyer where is_reviewed is false
29 const entries = await sellerToReview.findAll({
30   where: {
31     buyer_id,
32     is_reviewed: false
33   },
34   include: [
35     { model: user, as: 'seller', attributes: ['id', 'username'] },
36     { model: productlisting, as: 'product', attributes: ['id', 'title', 'location'],
37       include: [
38         { model: mediafile, as: 'mediafiles', attributes: ['file_path'], required: false, limit: 1 }
39       ]
40     }
41   ]
42 });

```

Figure 3: seller-review.controller.js Code 3

```

43 |
44 | // For each entry, get seller stats
45 | const result = await Promise.all(entries.map(async entry => {
46 |   const stats = await getSellerReviewStats(entry.seller_id);
47 |   return {
48 |     id: entry.id,
49 |     seller_id: entry.seller_id,
50 |     seller_name: entry.seller?.username,
51 |     product_id: entry.product_id,
52 |     product_name: entry.product?.title,
53 |     location: entry.product?.location,
54 |     purchase_date: entry.purchase_date,
55 |     media_file: entry.product?.mediafiles && entry.product.mediafiles.length > 0 ? entry.product.mediafiles[0].file_path : null,
56 |     seller_average_rating_value: stats.avg,
57 |     seller_total_rating: stats.total
58 |   };
59 | });
60 |

```

Figure 4: seller-review.controller.js Code 4

```

86 | // Create seller_review record
87 | const review = await sellerReview.create({
88 |   reviewee_id,
89 |   reviewer_id,
90 |   to_review_id,
91 |   review_value,
92 |   review_comment: review_comment || null
93 | });

```

Figure 5: seller-review.controller.js Code 5

4.6. Review - ChatHeader.jsx

- **Code Reviewer:** Hamza Butt
- **Code Owner:** Karan Patel
- **Date:** 27.06.2025
- **File to be reviewed:** UI/src/components/Chat/ChatHeader.jsx

Suggestions:

Extract this array into a shared constants file. Keeping “commonReasons” here makes it harder to update or reuse.

```

// Common report reasons for quick selection
const commonReasons = [
  'Inappropriate behavior',
  'Spam or scam',
  'Harassment',
  'Fake profile',
  'Inappropriate content'
];

```

Move localStorage helpers (getReportedUsers, addReportedUser, hasReportedUser) into a separate file (useReportUtils). This will be helpful to maintain single responsibility.


```
// Utility functions for managing reported users (consolidated)
const getReportedUsers = (currentUserId) => {
  try {
    const reportedUsers = localStorage.getItem(`reportedUsers_${currentUserId}`);
    return reportedUsers ? JSON.parse(reportedUsers) : [];
  } catch (error) {
    console.error('Error getting reported users:', error);
    return [];
  }
};

const addReportedUser = (currentUserId, reportedUserId) => {
  try {
    const reportedUsers = getReportedUsers(currentUserId);
    if (!reportedUsers.includes(reportedUserId)) {
      reportedUsers.push(reportedUserId);
      localStorage.setItem(`reportedUsers_${currentUserId}`, JSON.stringify(reportedUsers));
    }
  } catch (error) {
    console.error('Error adding reported user:', error);
  }
};

const hasReportedUser = (currentUserId, userIdToCheck) => {
  const reportedUsers = getReportedUsers(currentUserId);
  return reportedUsers.includes(userIdToCheck);
};
```

This is pure validation logic. pull it out into its own file like chatHeaderValidation.js. This helps to keep the component UI focused.

```
const validateReportReason = (reason) => {
  const trimmedReason = reason?.trim() || '';

  if (trimmedReason.length < 5) {
    return {
      isValid: false,
      message: 'Reason must be at least 5 characters long'
    };
  }

  if (trimmedReason.length > 500) {
    return {
      isValid: false,
      message: 'Reason must be less than 500 characters'
    };
  }

  return {
    isValid: true,
    message: 'Valid reason'
  };
};
```

Replace the number “3” with an enum or named constant, e.g.
 const PRODUCT_STATUS = { AVAILABLE: 1, REJECTED: 2, SOLD: 3 };

```
const isProductSold = productStatus === 3;
```

All of the text can be moved to a constant file. It will make it more readable and easy to modify.


```

const handleConfirmSold = async () => {
  if (!productId || !contact.id) {
    setSoldMessage('Error: Missing product or buyer information.');
```

`return;`

```
  }

  setIsMarkingSold(true);
  setSoldMessage('');

  try {
    const response = await markProductAsSoldAPI(productId, contact.id);

    if (response.success) {
      setSoldMessage('Product marked as sold successfully! This will help other users know it\'s no longer av
      setTimeout(() => {
        setIsSoldDialogOpen(false);
        setSoldMessage('');
        window.location.reload();
      }, 2000);
    } else {
      setSoldMessage(response.error || 'Failed to mark product as sold. Please try again.');
```

`}`

```
  } catch (error) {
    setSoldMessage('An error occurred while marking the product as sold. Please try again.');
```

`finally`

```
  {
    setIsMarkingSold(false);
  }
};
```

Replace `window.location.reload()` (which reloads the entire SPA). This will put bad experience for users.

```

window.location.reload();
```

In the designing part, you can convert all of the design into sub components that would make it easier to understand and modify. Like `userIcon.jsx` etc.

```

{/* User Avatar */}
<div className="relative">
  <img
    src={contact.avatar}
    alt={contact.name}
    className="w-10 h-10 rounded-full object-cover"
  />
</div>
```

4.7. Review - chat.controller.js

- **Code Reviewer:** Karan Patel
- **Code Owner:** Hamza Butt
- **Date:** 27.06.2025
- **File to be reviewed:** Backend/controllers/chat.controller.js

Overview

This module handles core chat functionalities:

- Fetching user-specific chats (`getUserChats`)
- Fetching messages in a specific chat (`getChatMessages`)
- Deleting a chat for a user (`deleteChat`)

Code Features

- **Query logic:** The conditional Op.or and Op.and blocks in `getUserChats` are
- **Soft delete strategy:** Dual-flag logic for `owner_deleted` and `other_deleted` is good — allows for per-user soft delete while enabling actual purge if both users delete.
- **Error handling:** Consistent try/catch blocks and `res.success/res.error` pattern — keeps responses uniform.

Suggestion / Improvement

Missing Null Checks:

`json.product` could be null if somehow the relation is broken. You use:

```
// attach a single image_url and strip out mediafiles array
const result = chats.map(c => {
  const json = c.toJSON();
  const media = json.product.mediafiles || [];
  json.product.image_url = media.length ? media[0].file_path : null;
  delete json.product.mediafiles;
  return json;
});
```

Muhammad Hamza Butt, last week • update chat implementataion logic ...

Suggestion:

```
const media = json.product?.mediafiles || [];
```

Consistent Logging:

Can add some logs for `getUserChats` as well in case of debugging some issue related to fetching chats.

Hard-coded status code:

Instead of hardcoded status you can create a constant for more readability:

```
    return res.success('User chats fetched successfully', result, 200);
  } catch (err) {
    return res.error(err.message || 'Failed to fetch user chats', 500);
  }
};

exports.getChatMessages = async (req, res) => {
  try {
    const { chat_id } = req.query;

    const chatExists = await chat.findByPk(chat_id);
    if (!chatExists) {
      return res.success('No messages yet. Chat can be started.', [], 200);
    }
  }
}
```

Suggestion:

```
const HTTP_STATUS = {
  OK: 200,
  BAD_REQUEST: 400,
  FORBIDDEN: 403,
  NOT_FOUND: 404,
  INTERNAL_SERVER_ERROR: 500
};
```

Optimize Database Query:

Consider adding indexes and limiting data fetched, it will improve performance when there are a lot of messages to fetch.

```
const chats = await chat.findAll({
  attributes: ['chat_id', 'created_at', 'product_owner_id', 'other_person_id'],
  where: deletionFilter,
  include: [
    {
      model: productlisting,
      as: 'product',
      include: [{
        model: mediafile,
        as: 'mediafiles',
        attributes: ['file_path'],
        where: { is_approved: true },
        required: false
      }]
    },
    {
      model: message,
      as: 'lastMessage',
      attributes: ['message_id', 'content', 'sender_id', 'receiver_id', 'created_at']
    },
    { model: user, as: 'owner', attributes: ['id', 'username', 'email', 'createdAt'] },
    { model: user, as: 'otherPerson', attributes: ['id', 'username', 'email', 'createdAt'] }
  ],
  order: [['chat_id', 'DESC']]
});
```

Muhammad Hamza Butt, 4 weeks ago • work on structuring product listing response an...

5. Self-check on best practices for security

Major assets we are protecting:

- User accounts & credentials (email, password)
- Product listings and user-generated content
- Internal messaging data between buyers and sellers
- Admin dashboard & moderation tools
- Database and API endpoints

Major threats & our protections:

Asset	Threat	Protection
User accounts & passwords	Credential theft, brute force attacks	Passwords are hashed using bcrypt before storage in the database (confirm: we never store plain-text passwords).
Product listings & content	Injection attacks, spam, or malicious data	All form inputs (title, description, tags, etc.) are validated on both frontend and backend. Special characters are escaped in database queries to prevent SQL/NoSQL injection.
Internal messaging data	Data leakage, XSS	Messages are sanitized before storage and rendering. HTML/JS injection is prevented using output encoding.
Admin tools	Unauthorized access	Role-based access control checks (eg, admin@hs-fulda.de is configured in code and ideally moved to env config). Admin-only API routes are secured by middleware checking user roles.

Database/API	DDoS, unauthorized queries	Backend API uses JWT auth to validate logged-in users. Query rate limiting could be applied to endpoints. The DB is hosted in a private subnet (AWS RDS) and cannot be reached publicly.
--------------	----------------------------	--

Confirm encryption of passwords:

All user passwords are securely hashed with bcrypt before saving in the database. No plain-text passwords are stored.

Confirm input data validation:

- On *Create Listing* page: validate title, description, price, tags, images, category.
- *Search bar* input is validated to allow up to **40 alphanumeric characters only** to prevent injection and maintain performance.
- Numeric fields (eg, price) are validated to ensure they contain valid numbers within a reasonable range.

These validations are implemented in the React frontend and double-checked in the Node.js/Express backend before database writes.

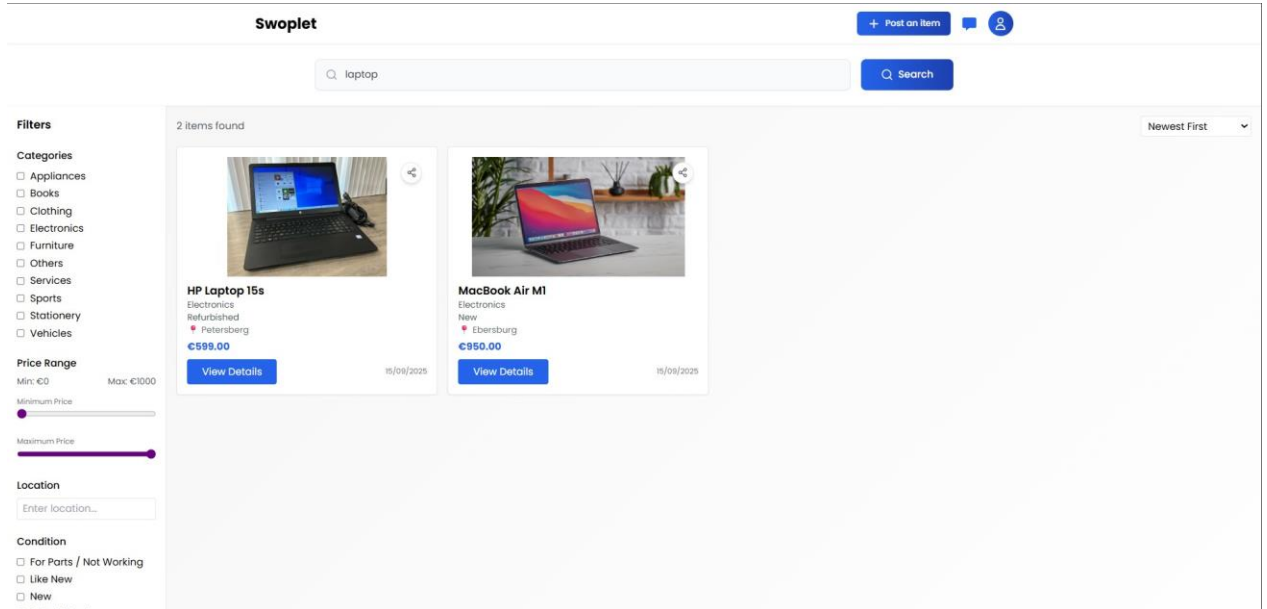
6. Self-check: Adherence to original Non-functional specs

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in Milestone 0. Application delivery shall be from chosen cloud server (**Achieved**)
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers. (**Achieved**)
3. All or selected application functions must render well on mobile devices. (**Achieved**)
4. Data shall be stored in the database on the team's deployment cloud server. (**Achieved**)
5. No more than 50 concurrent users shall be accessing the application at any time. (**Achieved**)
6. Privacy of users shall be protected and all privacy policies will be appropriately communicated to the users. (**Not Achieved**)
7. The language used shall be English (no localization needed). (**Achieved**)
8. Application shall be very easy to use and intuitive. (**Achieved**)
9. Application should follow established architecture patterns. (**Achieved**)
10. Application code and its repository shall be easy to inspect and maintain. (**Achieved**)
11. Google analytics shall be used (optional for Fulda teams) (**Not Achieved**)
12. No email clients shall be allowed. (**Achieved**)
13. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI. (**Achieved**)
14. Site security: basic best practices shall be applied (as covered in the class) for main data items. (**Achieved**)
15. Application shall be media rich (images, video etc.). Media formats shall be standard as used in the market today. (**Achieved**)
16. Modern SE processes and practices shall be used as specified in the class, including collaborative and continuous SW development. (**Achieved**)

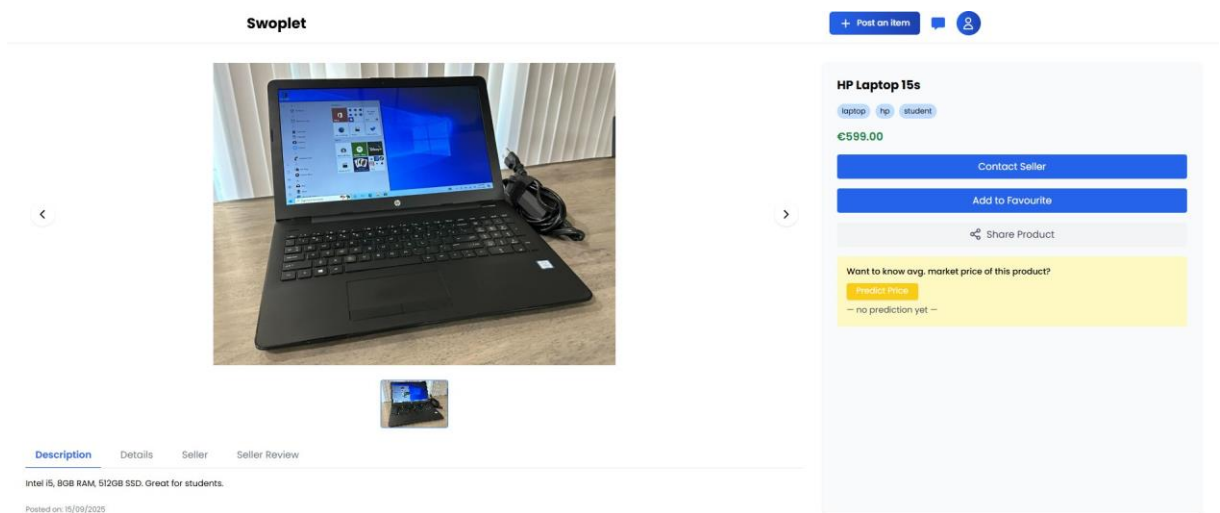
17. For code development and management, as well as documentation like formal milestones required in the class, each team shall use their own GitHub to be set-up by class instructors and started by each team during Milestone 0. (Achieved)
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "Fulda University of Applied Sciences Software Engineering Project, Summer 2025 For Demonstration Only" at the top of the WWW page. (Important so as to not confuse this with a real application). (Achieved)

Final Project Screenshots

1) Search Bar / Search Result Page



2) Product Detail Page



3) Messaging Page

